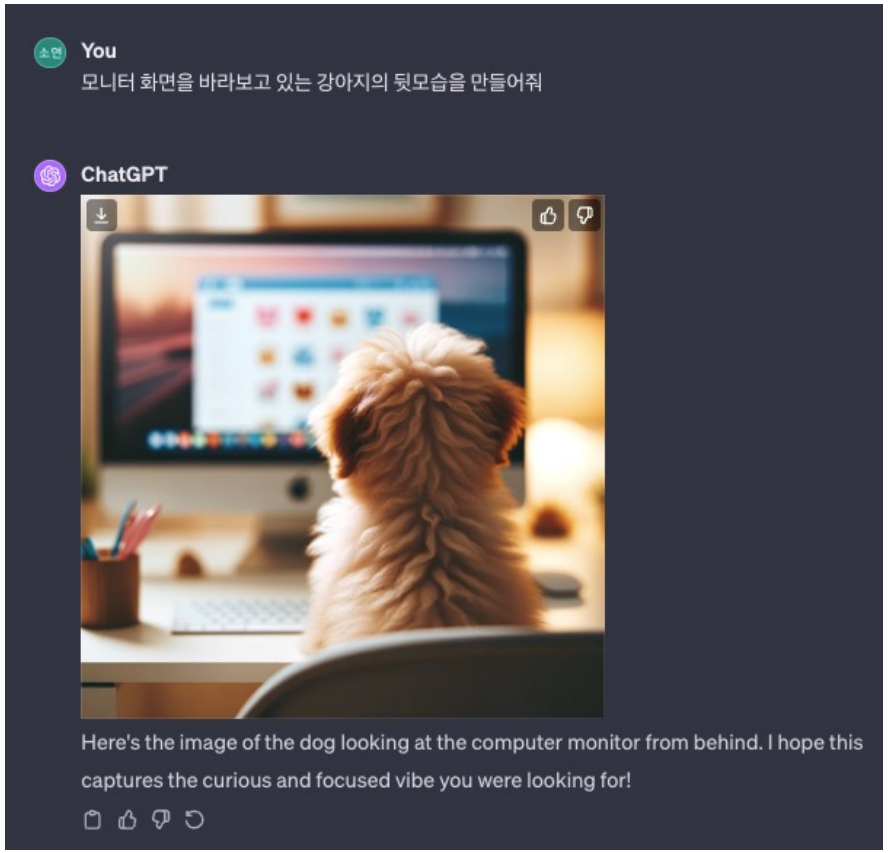
Generative Model

01

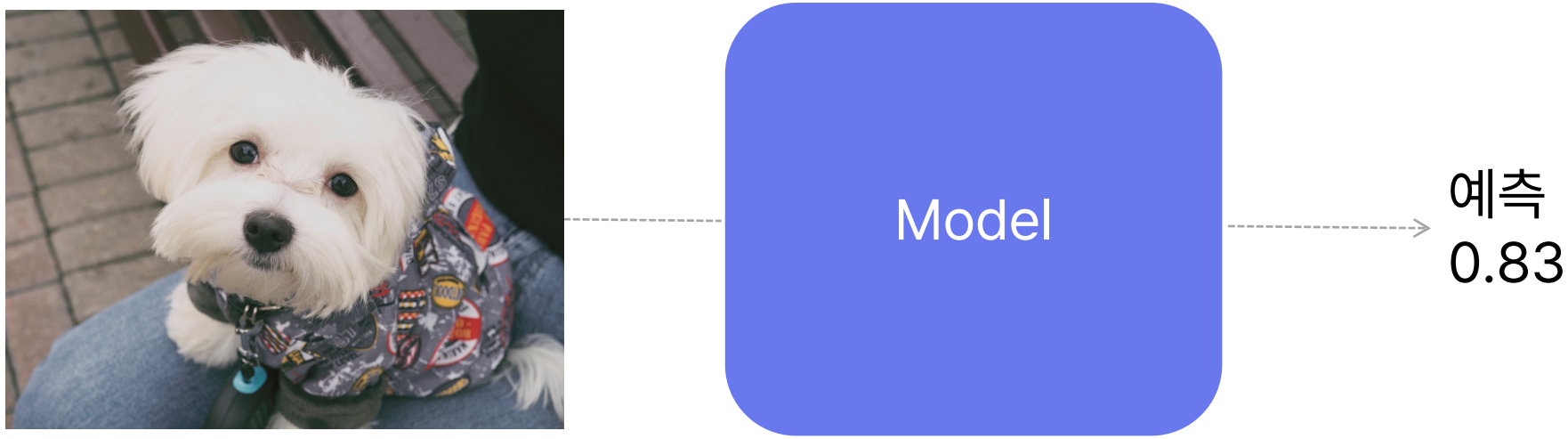
Generative Modeling

생성 모델링(Generative Modeling)

: 주어진 데이터셋과 **유사한 데이터를 생성하도록 모델을 훈련**하는 머신러닝의 한 분야



Q. 이 사진 속 동물이 강아지인가요?



판별 모델(Discriminative Model)

Q. 데이터셋과 유사하지만, 기존 데이터에 존재하지 않는 새로운 강아지 이미지를 만드는 모델



Model



생성 모델 (Generative Model)

판별 모델을 자세히 살펴봅시다.

1. 판별 모델의 데이터셋의 구성은?

2. 모델은 무엇을 학습하나요?

3. 결과값은 어떻게 나오나요?

판별 모델

훈련데이터


1

0

1

1

0

0

샘플

레이블

훈련

Model

(훈련데이터가
충분히 많다면)

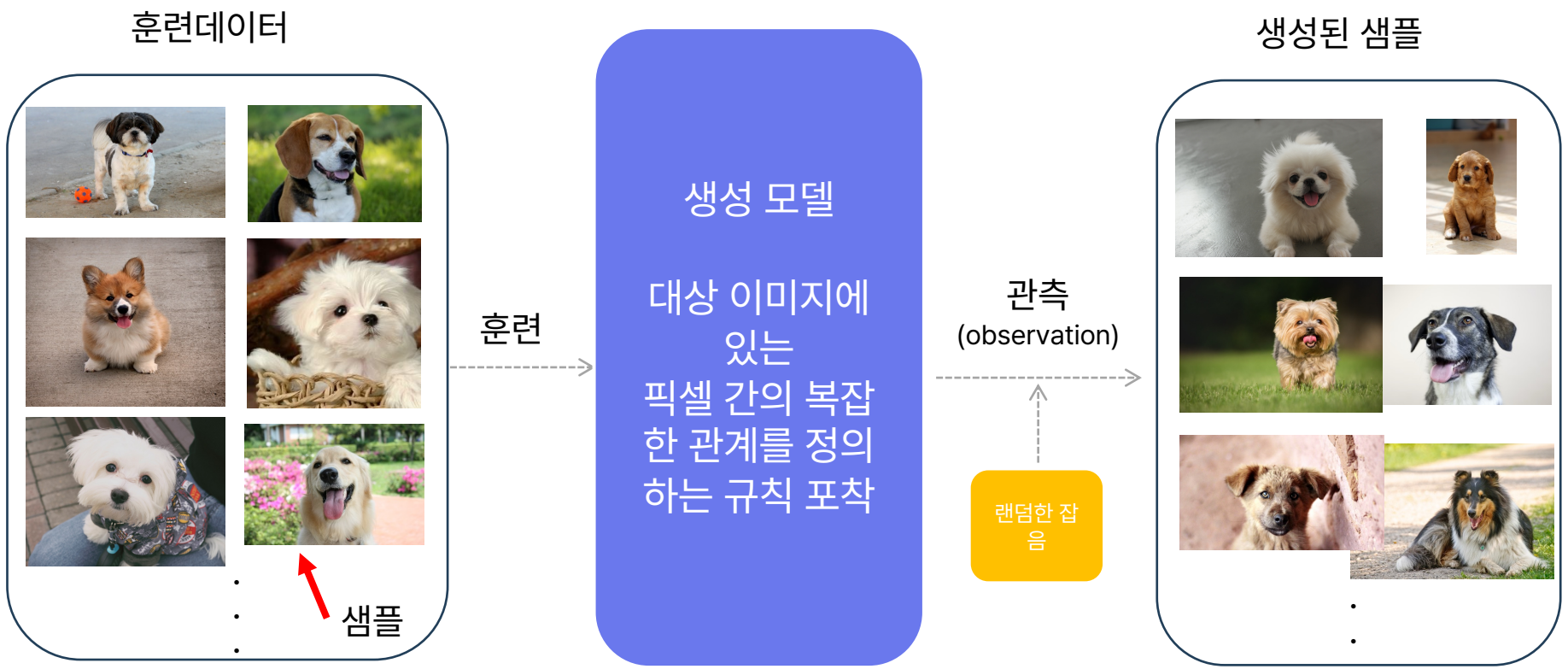
강아지임을 알아내는 특정 색깔, 형태, 질감, 구성 등을 학습

예측



강아지
가능성
0.83

생성 모델


훈련데이터
샘플
훈련
생성 모델
대상 이미지에 있는 픽셀 간의 복잡한 관계를 정의하는 규칙 포착
관측
(observation)
랜덤한 잡음
생성된 샘플

생성 모델의 특징

- 생성 모델을 만들려면 생성하려는 개체의 샘플이 많은 데이터셋이 필요합니다 (훈련데이터)

- 각 샘플은 많은 특성(feature)으로 이루어집니다.
  - 이미지 : 개별 픽셀값
  - 텍스트 : 개별 단어 또는 문자의 그룹

- 목표 : 일련의 새로운 특성을 생성하는 모델을 만드는 것
  - ➔ 원본 데이터와 동일한 규칙으로 생성된 것처럼 보이는 특성 만들기
  - ➔ 문제는 개별 픽셀 값이 가질 수 있는 경우의 수가 너무 많다!

만약 훈련데이터 안에 있는 모든 픽셀값의 평균값을 구했더니
아래와 같은 그럴듯한 이미지가 생성되었습니다.
이 모델은 생성모델인가요?



생성 모델의 특징

- 생성모델은 “확률적”이다.
  즉, 매번 동일한 값을 출력하는 것이 아니라 다양한 출력 결과를 관측(샘플링)할 수 있어야 합니다.

- 어떤 이미지는 훈련 데이터셋에 있을 것 같고, 다른 이미지는 그렇지 않은 이유를 설명하는 알려지지 않는 확률분포가 있다고 가정합니다.

  → 해야 할 일은 최대한 이 분포에 가깝게 흉내내는 모델을 만드는 것입니다.

  → 그 다음 이 분포에서 샘플링하여 원본 훈련 세트에 있을 것 같은 그러나 완전히 새로운 또 다른 샘플을 생성합니다.

생성 모델과 판별 모델

판별 모델 : 샘플 x가 주어졌을 때 레이블 y의 확률을 모델링 하는 것이 목표

$$P(y|x)$$를 추정

생성 모델 : 샘플 x를 관측할 확률을 모델링 하는 것이 목표

$$P(x)$$를 추정

생성 모델 훈련 → 샘플링 → 원본 훈련 데이터 셋에 속할 가능성이 높은 이미지 생성

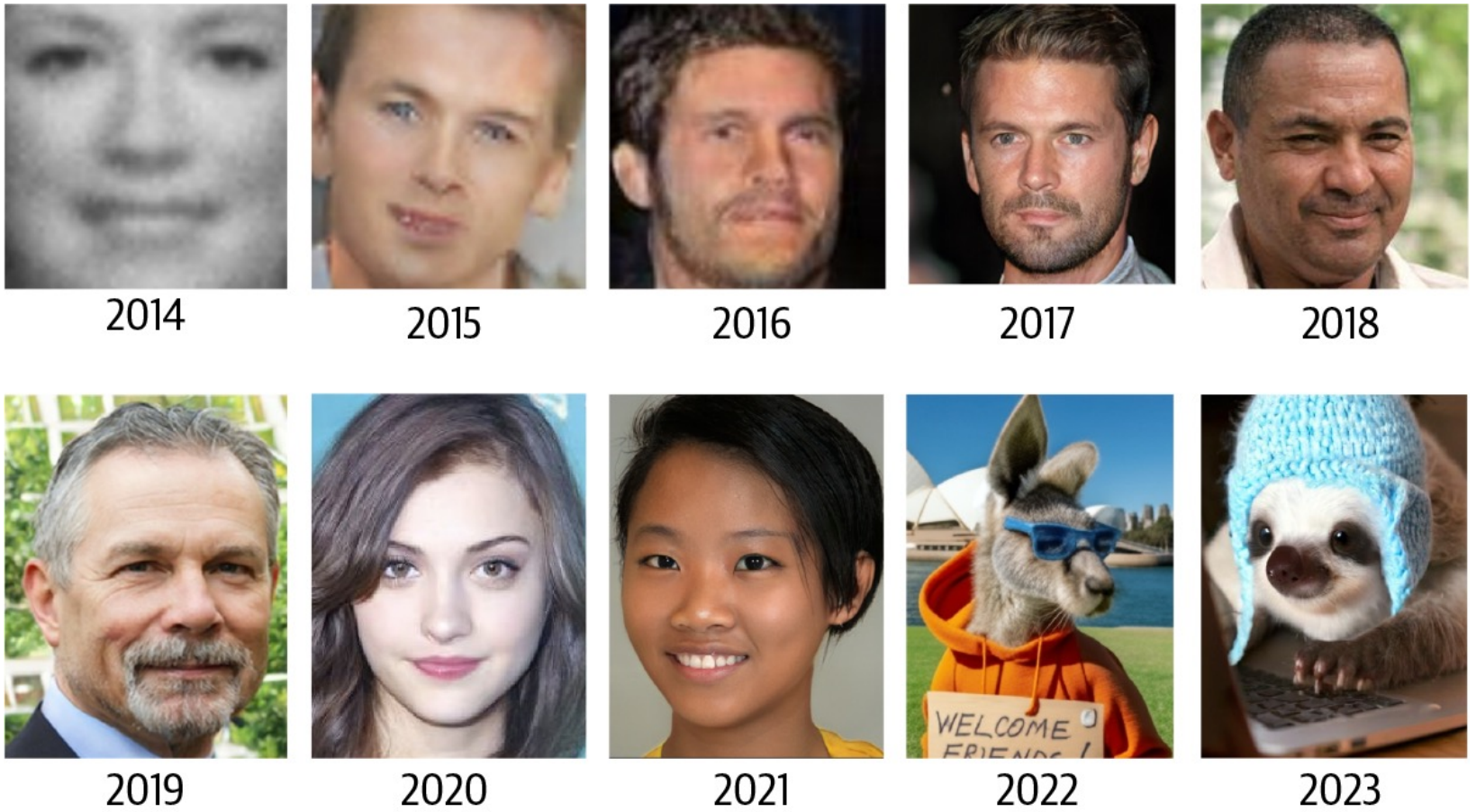
Q. 여러 종류의 과일이 있는 데이터셋으로 훈련한 생성모델이 “사과” 이미지를 생성하도록 요청하고 싶다면?

(레이블 y, 샘플 x)

$$P(x \mid y)$$

생성 모델과 판별 모델

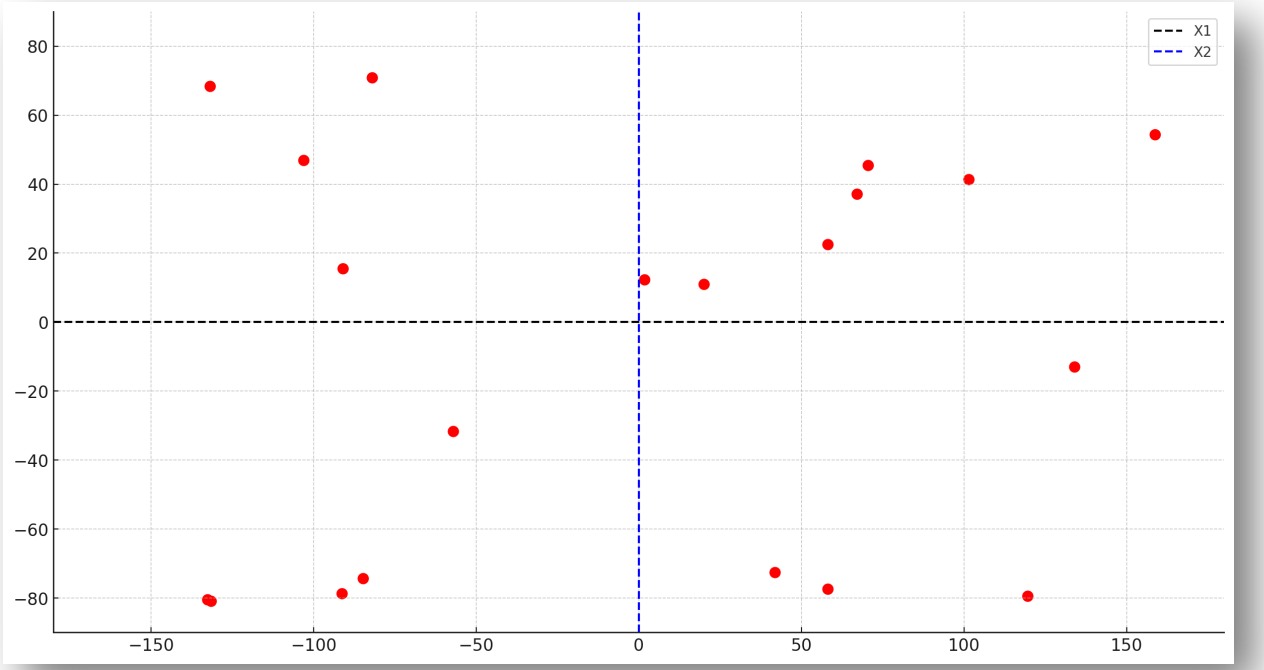
지난 10년간 크게 발전한 생성 모델링



https://www.oreilly.com/library/view/generative-deep-learning/9781098134174/ch01.html

간단한 생성 모델

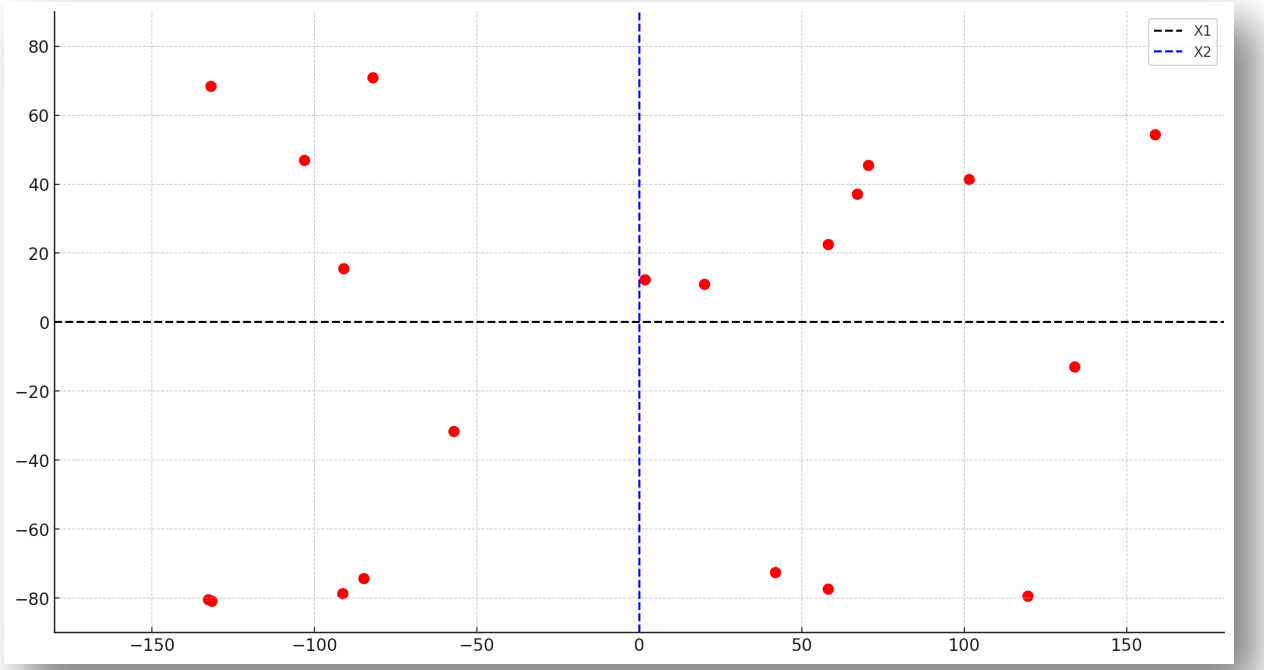
어떤 규칙(P_{data})이 있습니다. 이 규칙을 활용하여 포인트 집합 X를 아래 그림과 같이 생성했습니다.



같은 규칙으로 생성된 것 같은 또 다른 포인트 x = (x1, x2)를 선택하세요!

간단한 생성 모델

어디에 점을 찍으시겠습니까?
기존 점의 정보를 사용해 임의의 모델을 구성해보세요.



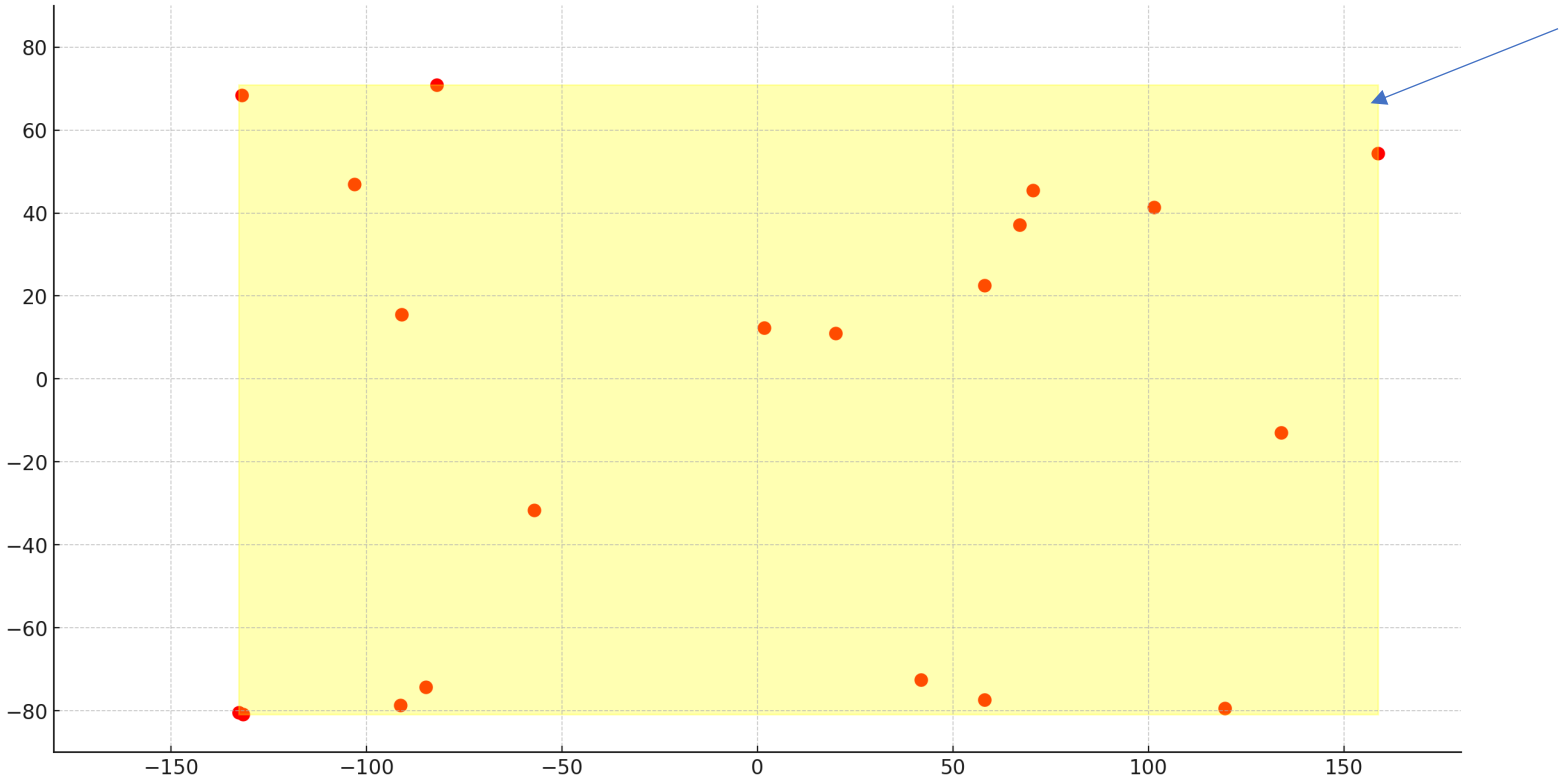
간단한 생성 모델

여러분이 만든 임의의 모델은 기존의 데이터를 토대로
"또 다른 점은 ~한 공간에서 발견될 가능성이 높다"라는 것을 말하는 모델입니다.

이러한 점에서 여러분의 모델 P_{model} 은 P_{data}의 추정입니다.

간단한 생성 모델

저는 P_{model} 로 이러한 사각형을 생각했습니다.
즉, 이 상자 바깥에는 빨간 점이 없고, 상자 안에만 존재한다고 생각했습니다.

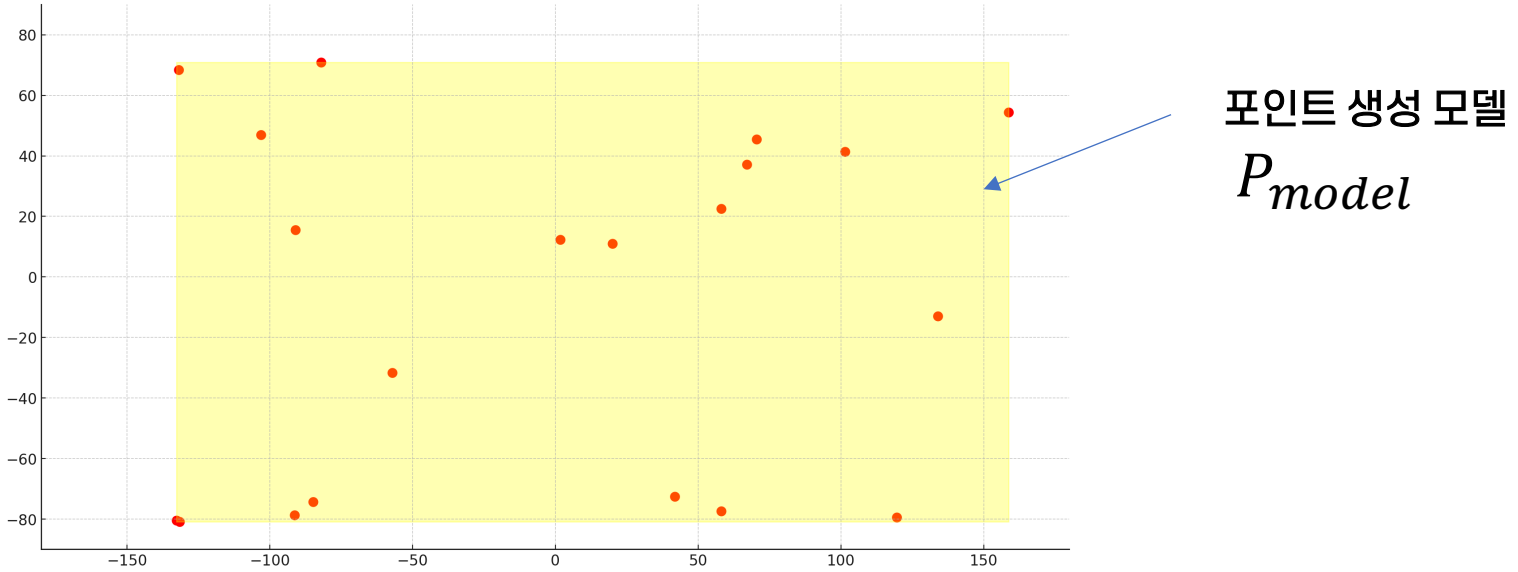


포인트 생성 모델

P_{model}

간단한 생성 모델

새로운 샘플을 생성하려면 이 상자 안에서 랜덤하게 한 점을 선택하면 됩니다.
이걸 "P_{model} 분포로부터 샘플링한다"라고 이야기 합니다.



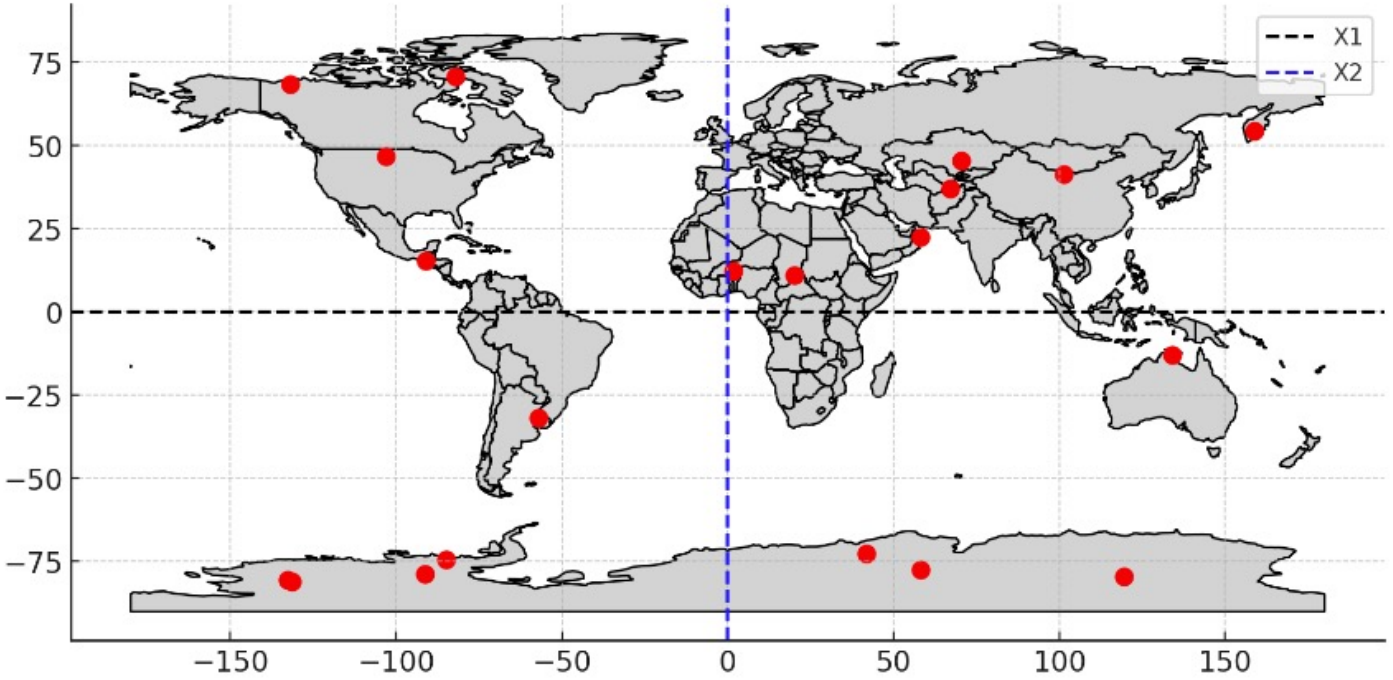
이제 우리는 첫번째 생성 모델을 만들었습니다.
훈련 데이터를 사용해 -> 훈련데이터에 속하지 않은 다른 점을 생성할 수 있는
모델(노란 상자)을 만들었습니다.

<생성 모델링 프레임워크>

- 샘플데이터셋 X를 가지고 있습니다.
- 샘플이 알려지지 않은 어떤 분포 P_{data} 로 생성되었다고 가정합니다.
- 우리는 P_{data}를 흉내내는 P_{model} 을 만들려고 합니다.
  목표는 P_{model}에서 샘플링하여 P_{data}에서 뽑은 것 같은 샘플을 생성하는 것입니다.
- 따라서 P_{model}의 바람직한 속성은 다음과 같습니다.
  - 정확도
    생성된 샘플의 P_{model}이 높으면 P_{data}에서 뽑은 것처럼 보여야 합니다.
    P_{model}이 낮으면 P_{data}에서 뽑은 것 처럼 보여서는 안됩니다.
  - 생성
    P_{model}에서 새로운 샘플을 쉽게 샘플링할 수 있어야 합니다.
  - 표현
    데이터의 다양한 고수준 특성이 P_{model}로 어떻게 표현되는지 이해할 수 있어야 합니다.

이 모델은 잘 만든 모델일까?

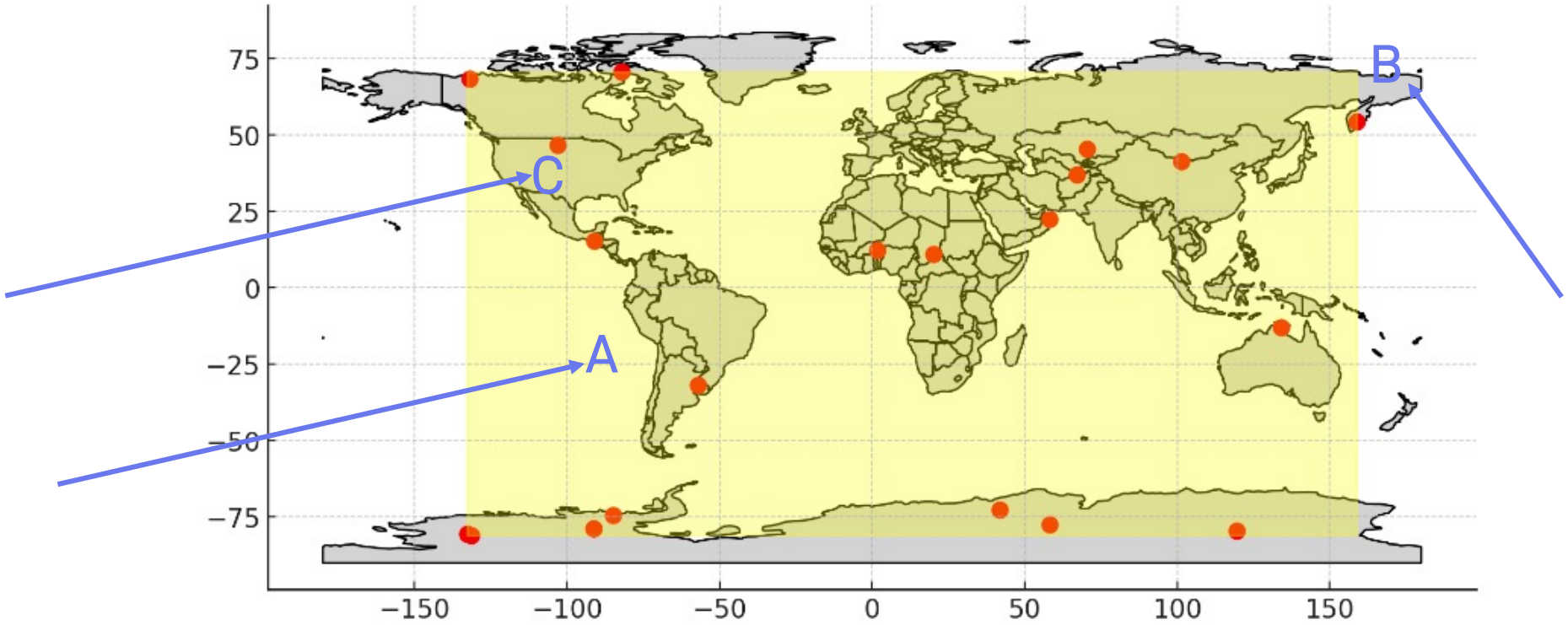
실제 데이터 생성 분포인 P_{data}를 공개합니다.



데이터 생성 규칙은 전 세계 대륙에 걸쳐서 선택된 단순한 균일분포입니다.

이 모델은 잘 만든 모델일까?

포인트 A, B, C를 살펴보고, 노란 상자 모델을 평가해보세요



이 모델은 잘 만든 모델일까?

완벽하진 않아도 노란색 상자에 대한 균등분포이기 때문에 샘플링하기 쉽습니다.
즉, 간단하게 생성할 수 있습니다.

노란색 상자 모델은 데이터 생성 규칙을 과하게 단순화했습니다.
즉, 이 모델은 복잡한 분포의 단순 표현이라고 말할 수 있습니다.

이 표현은 내재된 고수준 특성을 "일부" 포착할 수 있습니다.
예를 들어, 여러 개의 대륙 → 하나의 큰 대륙으로 단순화했다고 볼 수 있습니다.

표현 학습(Representation Learning)

딥러닝에서 고차원 데이터의 "표현(Representation)"을 배운다는 것은 중요한 주제입니다.

- 표현 학습(Representation Learning)은 원본 데이터(raw data)에서 중요한 특징(feature)을 자동으로 학습하여, 모델이 더 효과적으로 학습할 수 있도록 하는 기법입니다.
- 사람이 직접 특징을 설계하는 수동 특징 엔지니어링(Feature Engineering)과 대비됩니다.

표현학습의 주요 기법

- 신경망의 은닉층

  예) CNN의 초반 층은 가장자리를 학습하고, 후반 층은 객체의 형태를 학습

- 오토인코더

  입력 데이터를 압축하여 의미 있는 표현(latent representation)을 학습

표현 학습(Representation Learning)

생성 모델에서는 고차원 데이터의 “표현(Representation)”을 배운다는 것이 중요한 주제입니다.

우리가 몽타주를 그린다고 생각해봅시다.

유재석님을 모르는 사람에게
이 이미지에 대해서 여러분이 설명하면
몽타주를 그린다고 생각해봅시다.

어떻게 설명하면 좋을까요?



표현 학습(Representation Learning)

설명1.
- 안경을 쓰고 웃고 있다
- 입이 살짝 돌출형이다.
- 마르고 탄탄한 체격이다
- ...



설명2.
- 1x1 픽셀 RGB 값
- 1x2 픽셀 RGB 값
- ...
- 540x720 픽셀 RGB값

표현 학습(Representation Learning)

- 설명1.

  - 안경을 쓰고 웃고 있다
  - 입이 살짝 돌출형이다.
  - 마르고 탄탄한 체격이다
  - ...

- 이런 설명이 10개 정도 있으면 상대방은 그림으로 그려낼 수 있을 겁니다.
  이 이미지는 완벽하지는 않겠지만 유재석님을 티비에서 보고 구별할 수 있을 정도로 실제와 유사할 것입니다.

  이것이 표현학습(Representation Learning)에 깔려 있는
  핵심 아이디어입니다.



표현 학습(Representation Learning)

즉, 표현학습이란

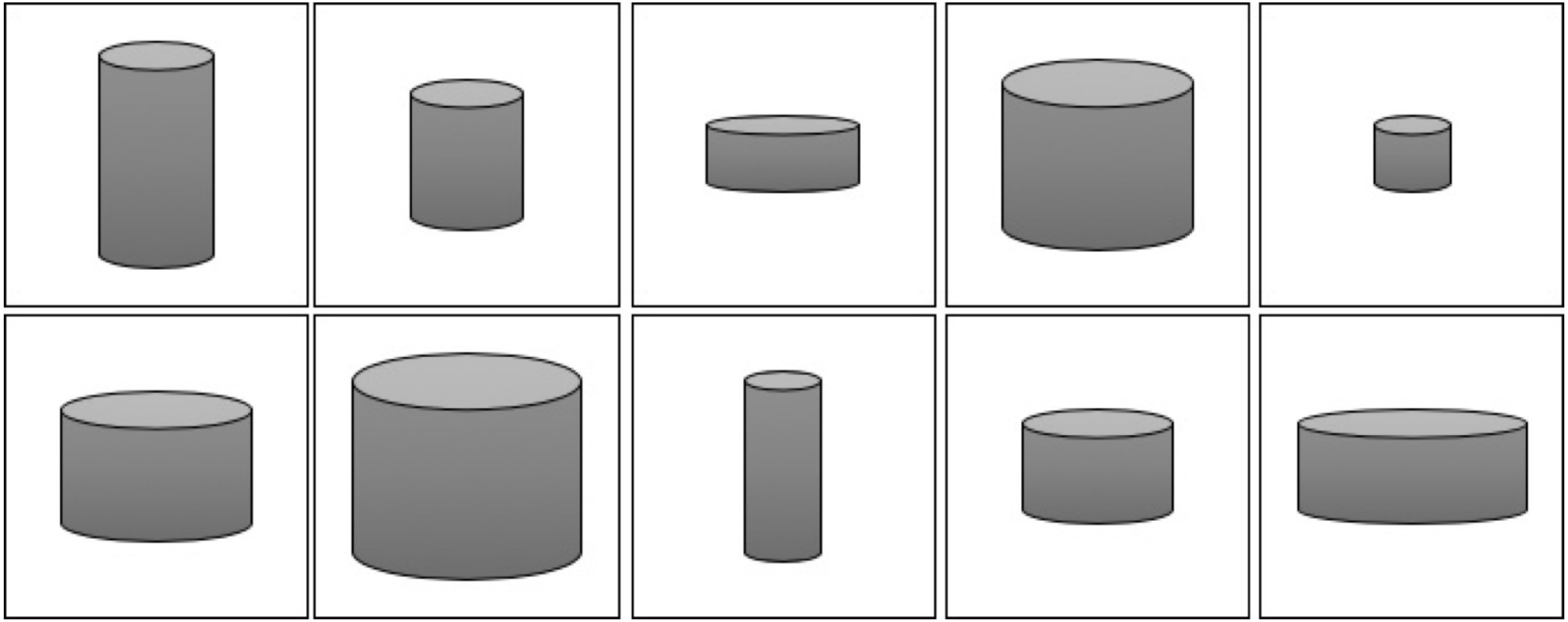
고차원 표본 공간을 직접 모델링 하는 방식이 아니라 대신

저차원의 잠재공간(latent space)을 사용해 훈련 세트의 각 샘플을 표현하고 이를 원본 공간의 포인트에 매핑합니다.

- 학습 데이터 잠재공간 생성 → 잠재공간의 각 포인트는 어떤 고차원 이미지에 대한 표현

표현 학습(Representation Learning)

원기둥 데이터셋



참고 자료: 데이비드 포스터, 만들면서 배우는 생성 AI (2판), 한빛미디어, 2022

이 데이터셋을 2가지 특성으로 표현하려면?

이 데이터셋을 저차원의 잠재공간으로 표현하려면?

잠재공간

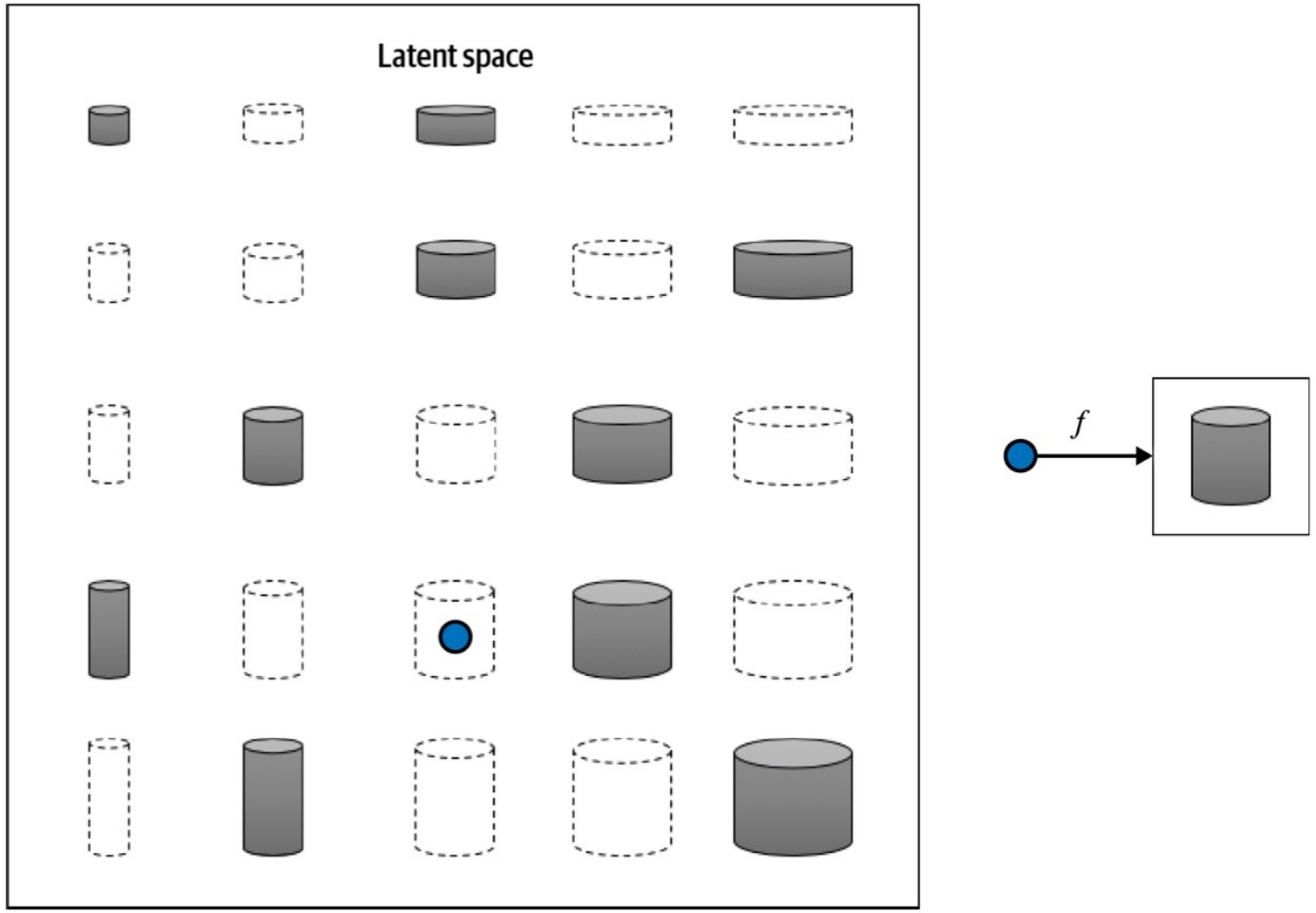


Figure 1-8. The 2D latent space of biscuit tins and the function f that maps a point in the latent space back to the original image domain

참고 자료: 데이비드 포스터, 만들면서 배우는 생성 AI (2판), 한빛미디어, 2022

표현 학습(Representation Learning)

훈련 세트의 이미지가 고차원 픽셀 공간으로 주워지더라도

각 원통 이미지를 2차원의 잠재 공간의 한 포인트로 변환할 수 있습니다.

또한 적절한 매핑함수 f를 잠재 공간의 새로운 포인트에 적용하여

훈련 세트에 없는 새로운 원기둥 이미지를 생성할 수도 있습니다.

의미 있는 잠재공간에서 표현 벡터를 조작하여 이미지의 고수준 속성에 영향을 미치는 연산을 수행할 수 있습니다.

원기둥의 높이를 살짝 키우고 싶을 때,

개별 픽셀을 하나하나 다 조정하는 것이 아니라

잠재공간의 높이 차원에 1을 더하고 매핑함수를 적용해 이미지를 얻습니다.

참고 자료: 데이비드 포스터, 만들면서 배우는 생성 AI (2판), 한빛미디어, 2022

표현 학습(Representation Learning)

원본 데이터 셋을 간단한 잠재공간으로 설명할 수 있음을 기계가 학습하는 것은 쉽지 않습니다.

“높이”와 “너비”가 이 데이터셋을 가장 잘 설명하는 두 개의 잠재공간 차원임을 알아야 합니다

이 공간의 한 포인트를 회색 원통에 매핑하는 매핑함수 f를 학습해야 합니다.

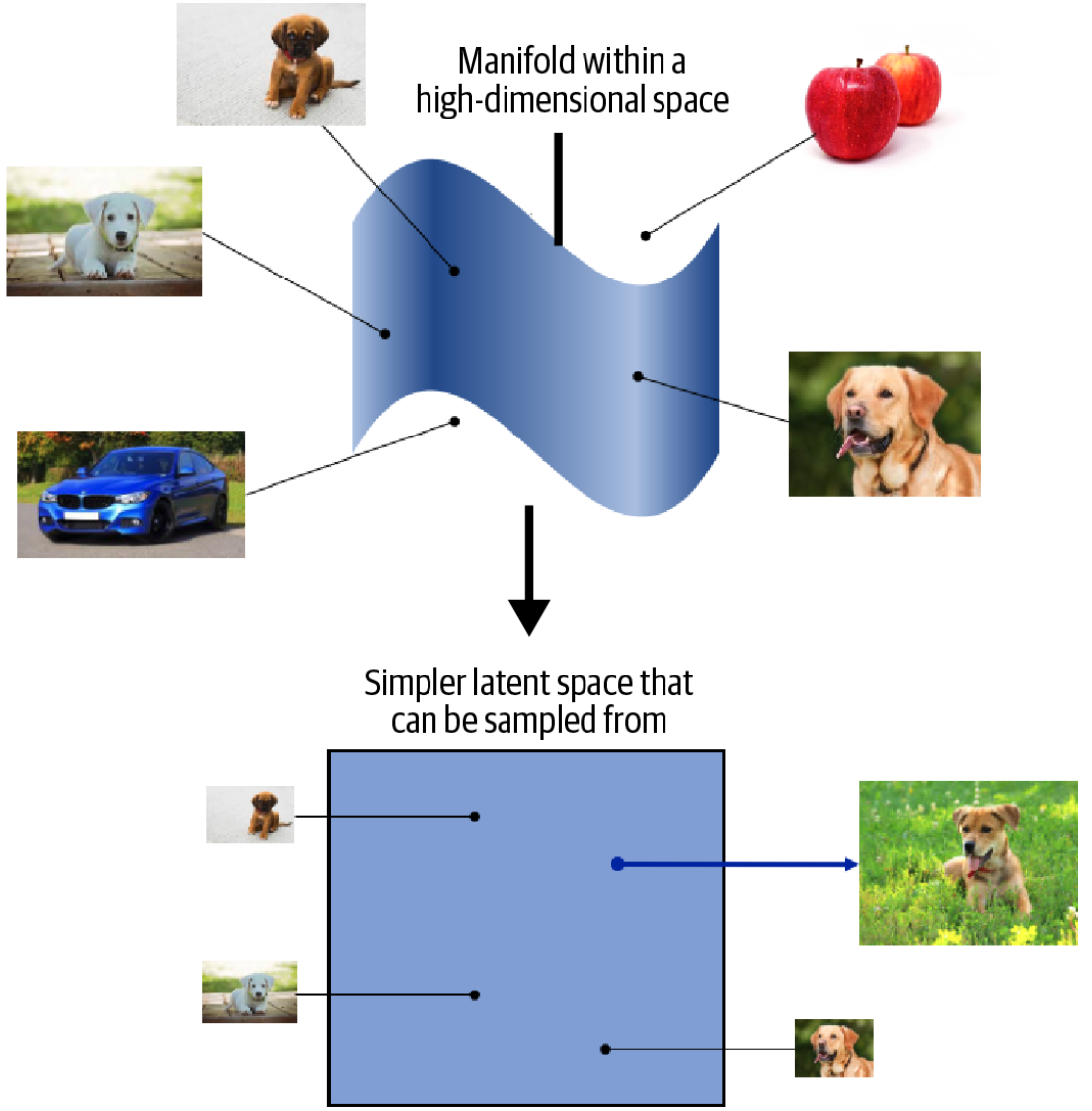
참고 자료: 데이비드 포스터, 만들면서 배우는 생성 AI (2판), 한빛미디어, 2022

1. 훈련 데이터셋을 잠재공간으로 인코딩

2. 잠재공간에서 샘플링

3. 샘플링 한 데이터를 디코딩하여
원래 도메인으로 되돌리기

인코더 – 디코더 기법
(많은 생성 모델링 기법에서 채택하는 기법)



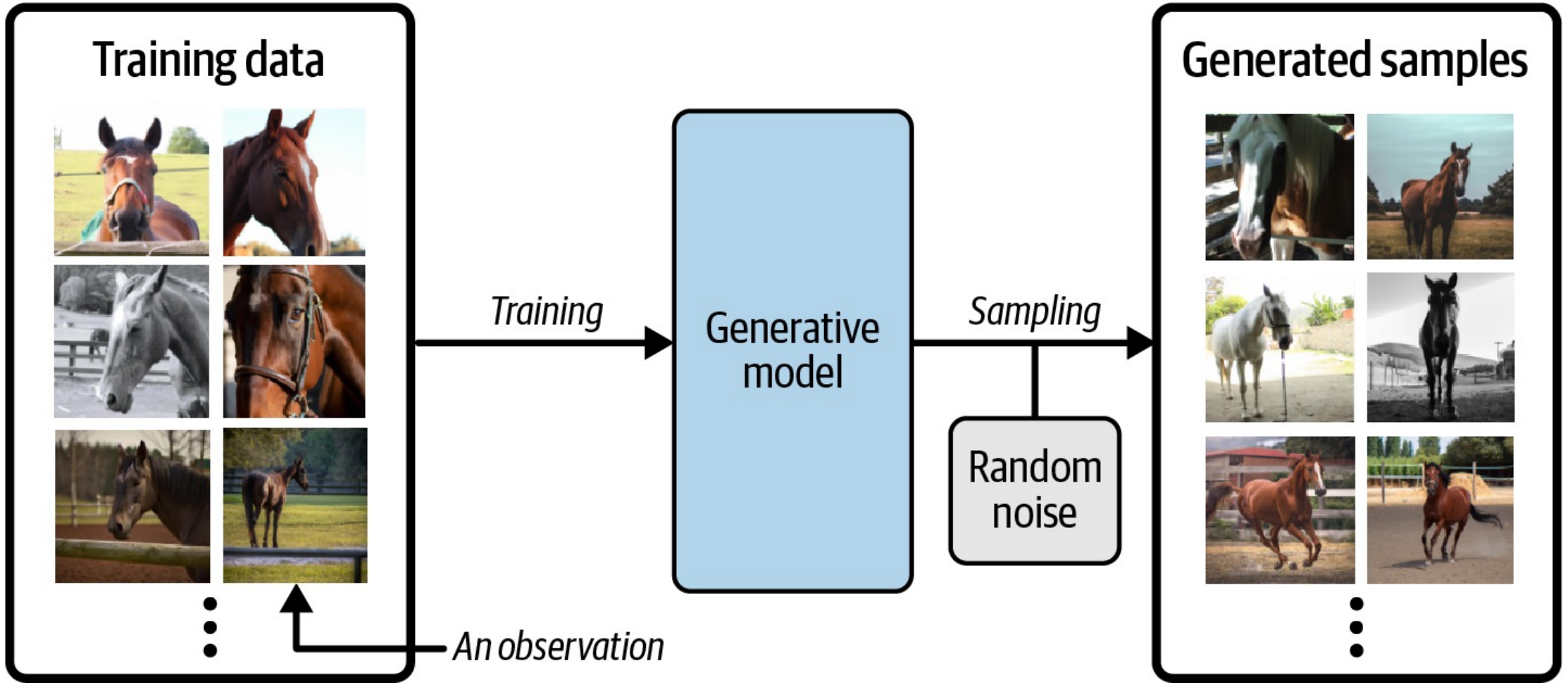
정리해봅시다

- 생성 모델(Generative Model)이란 데이터를 생성하는 방법을 학습하는 기계 학습 모델입니다.

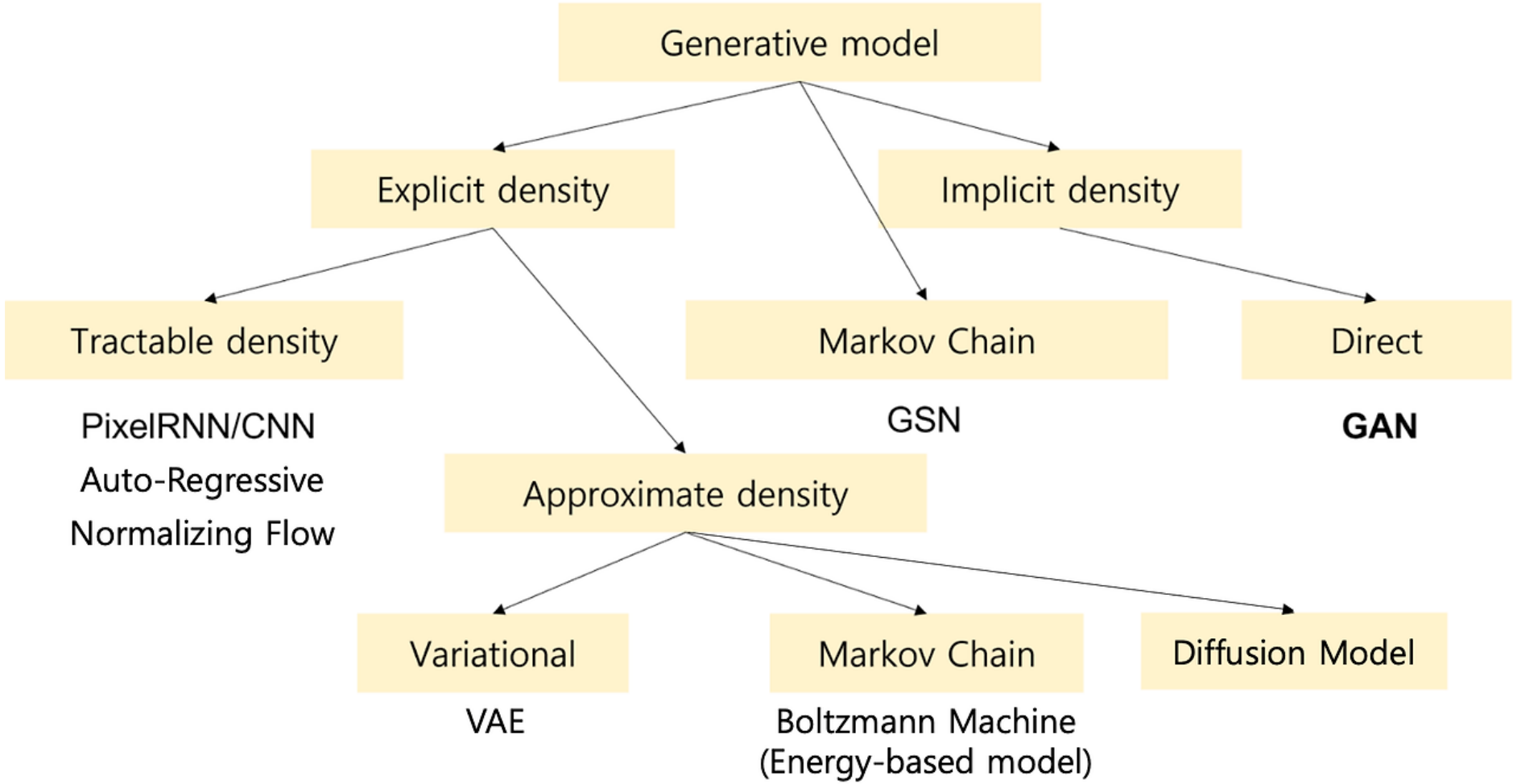
- 이 모델은 주어진 데이터 세트에서 데이터의 분포를 학습하고, 이를 바탕으로 새로운 데이터 인스턴스를 생성할 수 있습니다. 생성 모델의 주요 목적은 관찰된 데이터와 유사한, 그러나 실제로는 존재하지 않는 데이터를 만드는 것입니다.

- 샘플 x를 관측할 확률을 모델링 하는 것이 목표 → P(x)를 추정 → 생성 모델링 프레임워크

- 생성 모델을 잘 만들려면 고차원 데이터의 표현을 잘 배워야 합니다.
  표현 학습은 데이터로부터 의미 있는 특징을 추출하는 과정이며,
  잠재 공간은 이러한 특징들이 매핑되는 곳입니다


Training data
Training
Generative model
Sampling
Random noise
Generated samples
An observation

생성 모델링 방식 분류



https://chan4im.tistory.com/262

생성 모델링 방식 분류

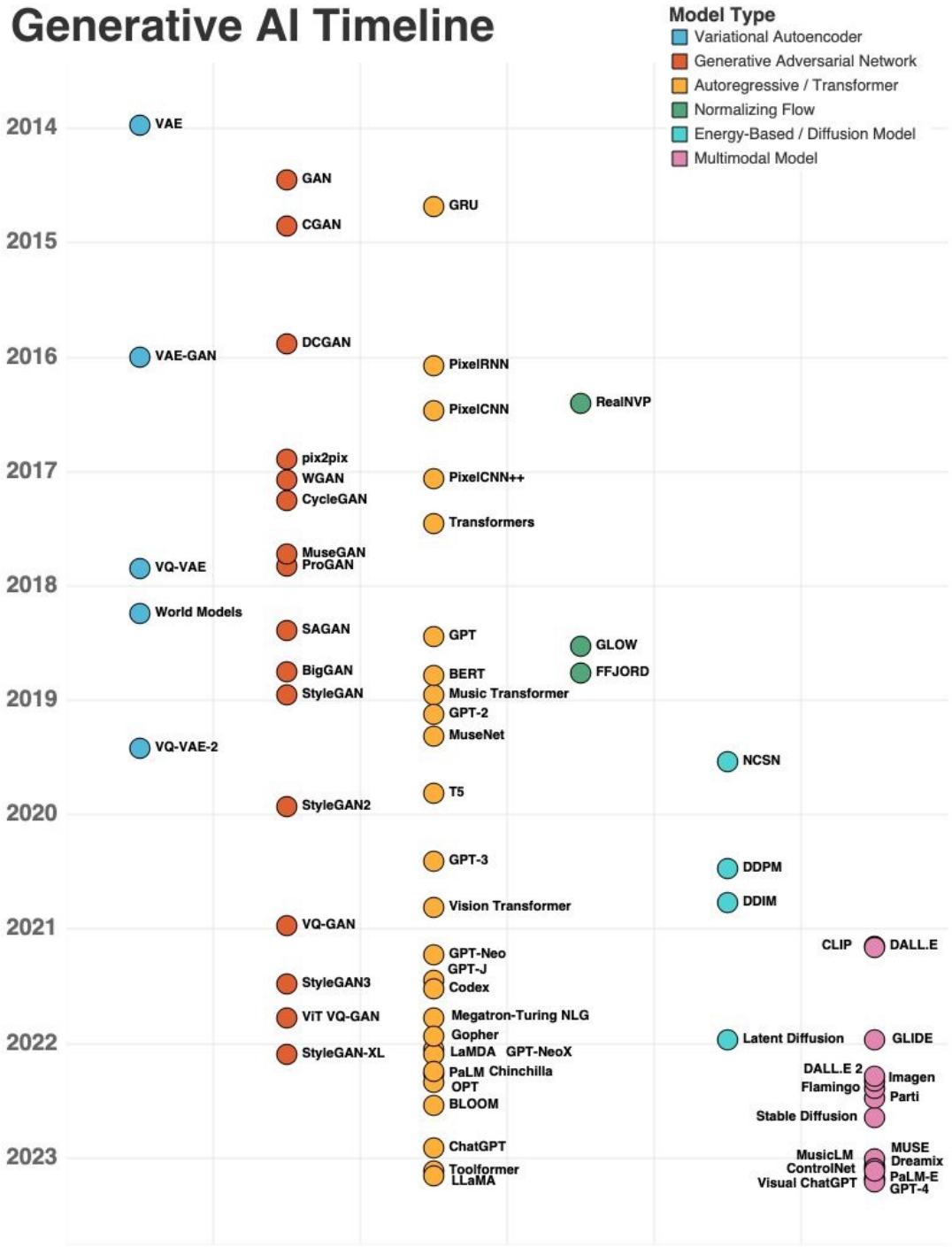
생성 모델이란

- 생성모델은 주어진 데이터의 분포를 학습하여, 새로운 데이터를 생성할 수 있는 모델입니다.
- 쉽게 말해, 생성모델은 기존의 데이터를 “이해”하고, 그 특성을 반영한 새로운 샘플을 만들어낼 수 있습니다.

생성 모델의 다양한 접근 방식

생성 모델이 밀도함수 $p_\theta(x)$ 를 모델링 하는 방식이 조금씩 다릅니다.

- 명시적으로 밀도함수를 모델링하지만 밀도함수를 다루기 쉽도록(즉, 계산할 수 있도록) 어떤 식으로 모델을 제약합니다.
- 다루기 쉬운 밀도 함수의 근사치를 명시적으로 모델링
- 데이터를 직접 생성하는 확률적 과정을 통해 밀도함수를 암묵적으로 모델링



https://www.kaggle.com/discussions/getting-started/397345

대표적인 생성 모델 기법

주요 모델

- VAE (Variational Autoencoder): 확률적 인코더-디코더 구조를 사용하여 데이터를 압축하고 재구성하는 방식
- GAN (Generative Adversarial Network): 생성자와 판별자가 경쟁하면서 더욱 정교한 데이터를 생성하는 방식
- Diffusion Model: 점진적으로 노이즈를 제거하면서 데이터를 생성하는 최근 연구에서 주목받는 방식

02

Variational Auto Encoder

Auto Encoder

- AutoEncoder(자동 인코더)는 입력 데이터를 저차원 표현(latent space)으로 압축하고, 다시 원본과 유사한 데이터를 복원하는 신경망 모델
- 라벨이 없는 학습 데이터에서 더 낮은 차원의 특징 표현(feature representation)을 학습하는 **비지도 학습 접근법(unsupervised approach)**
- 주로 **차원 축소, 특징 학습, 이상 탐지** 등에 활용

1. Auto Encoder 모델의 데이터셋의 구성은?

2. 모델의 목적은 무엇인가요?

3. 결과값은 어떻게 나오나요?




Autoencoder



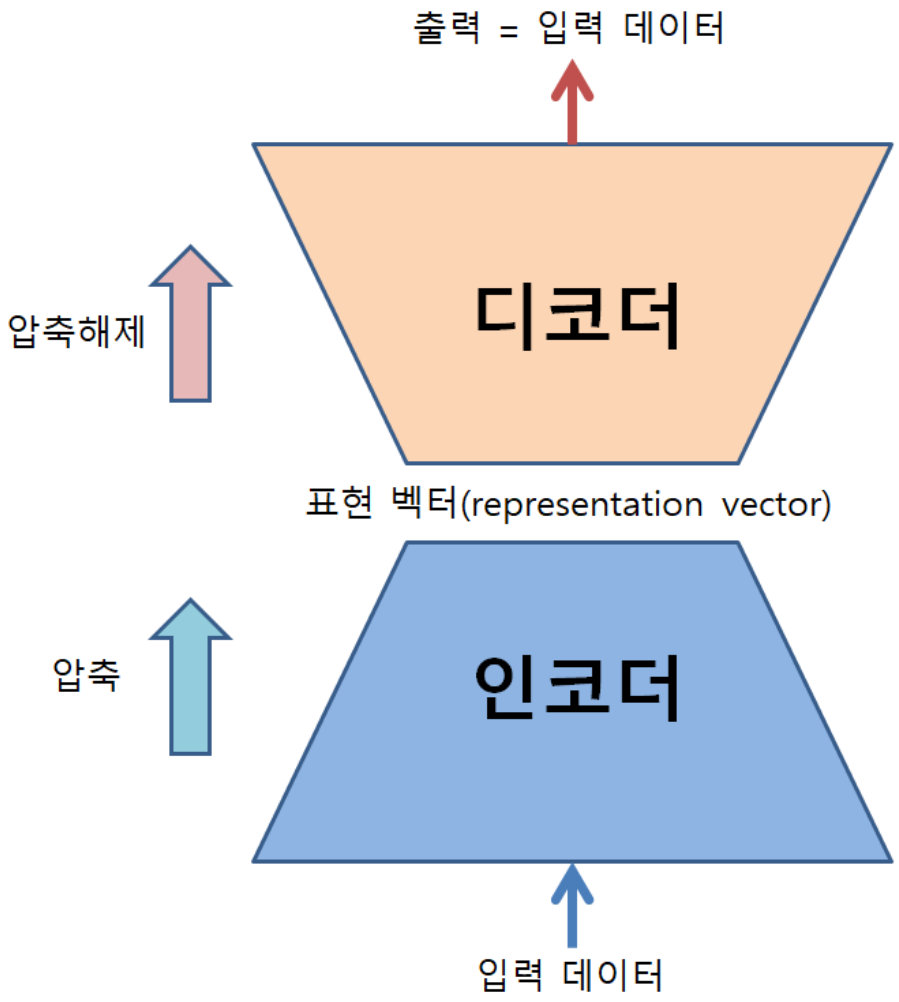
Encoder-Decoder 구조

Encoder:

- 데이터를 압축된 형태로 변환하는 역할을 수행
- 고차원 데이터를 저차원으로 변환하여 핵심 특징을 추출
- 예: 이미지를 입력받아, 해당 이미지의 중요한 특징만 포함된 벡터로 변환

Decoder:

- 압축된 표현을 다시 원래 데이터로 복원하거나, 새로운 형태로 변환
- Encoder의 결과물(잠재 표현)을 기반으로 데이터를 재구성
- 예: Latent Vector를 받아 이미지를 복원.



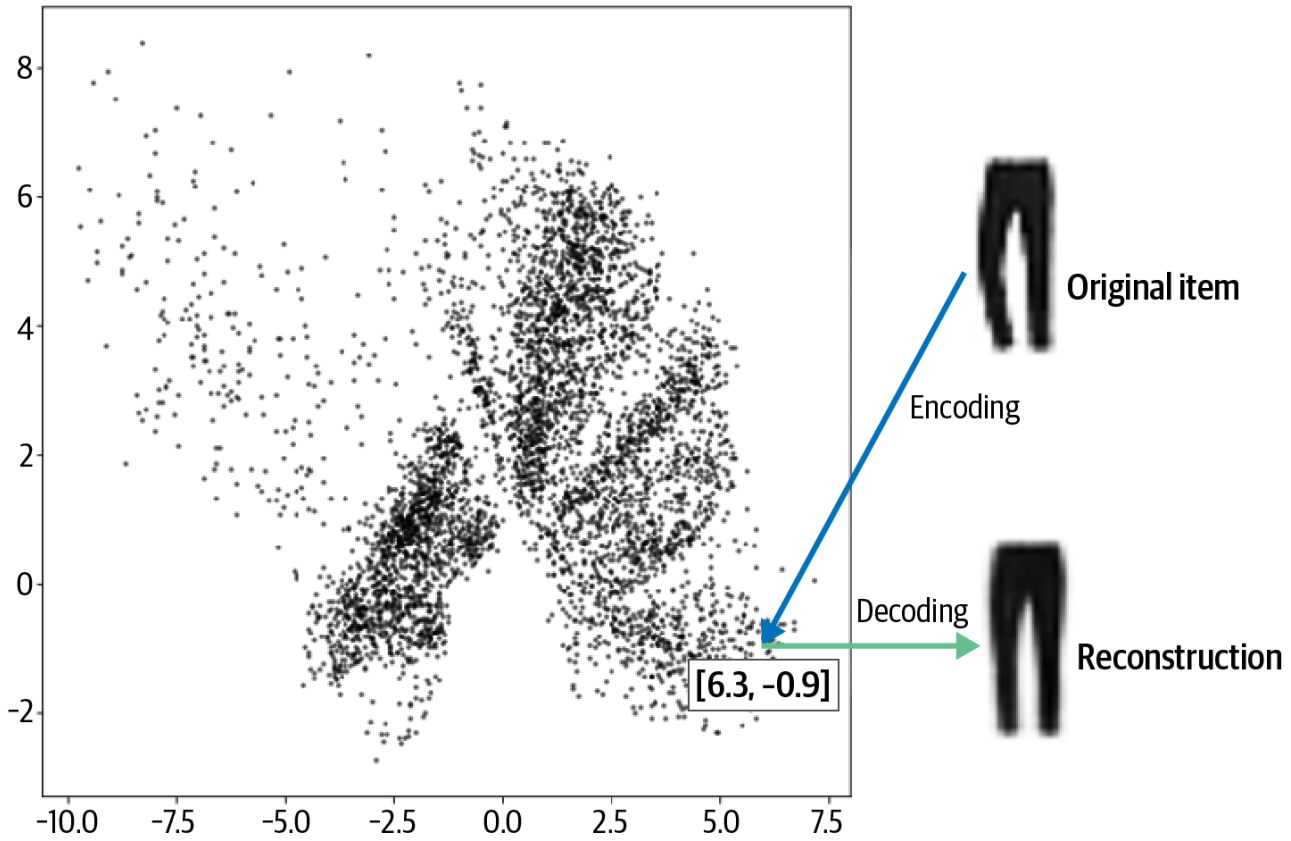
https://angeloyeo.github.io/2020/10/10/autoencoder.html#google_vignette

- **2D의 무한한 공간에 온갖 종류의 옷들을 나타낸다고 해봅시다.**

인코딩
: 각각의 옷을 특정 위치로 이동시키는 과정

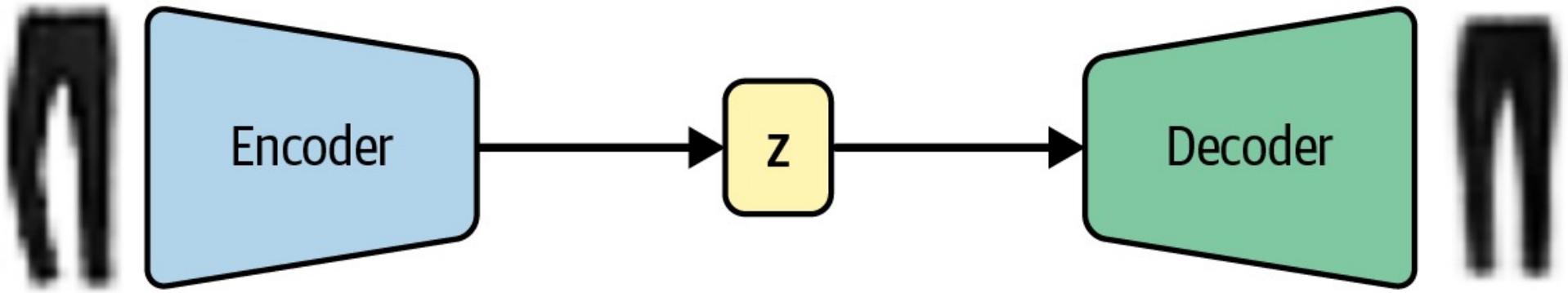
디코딩:
위치(벡터)로부터 옷을 생성하는 과정

임베딩 :
고차원 데이터(옷 이미지)를
저차원의 벡터 공간(점)으로 표현하는 과정



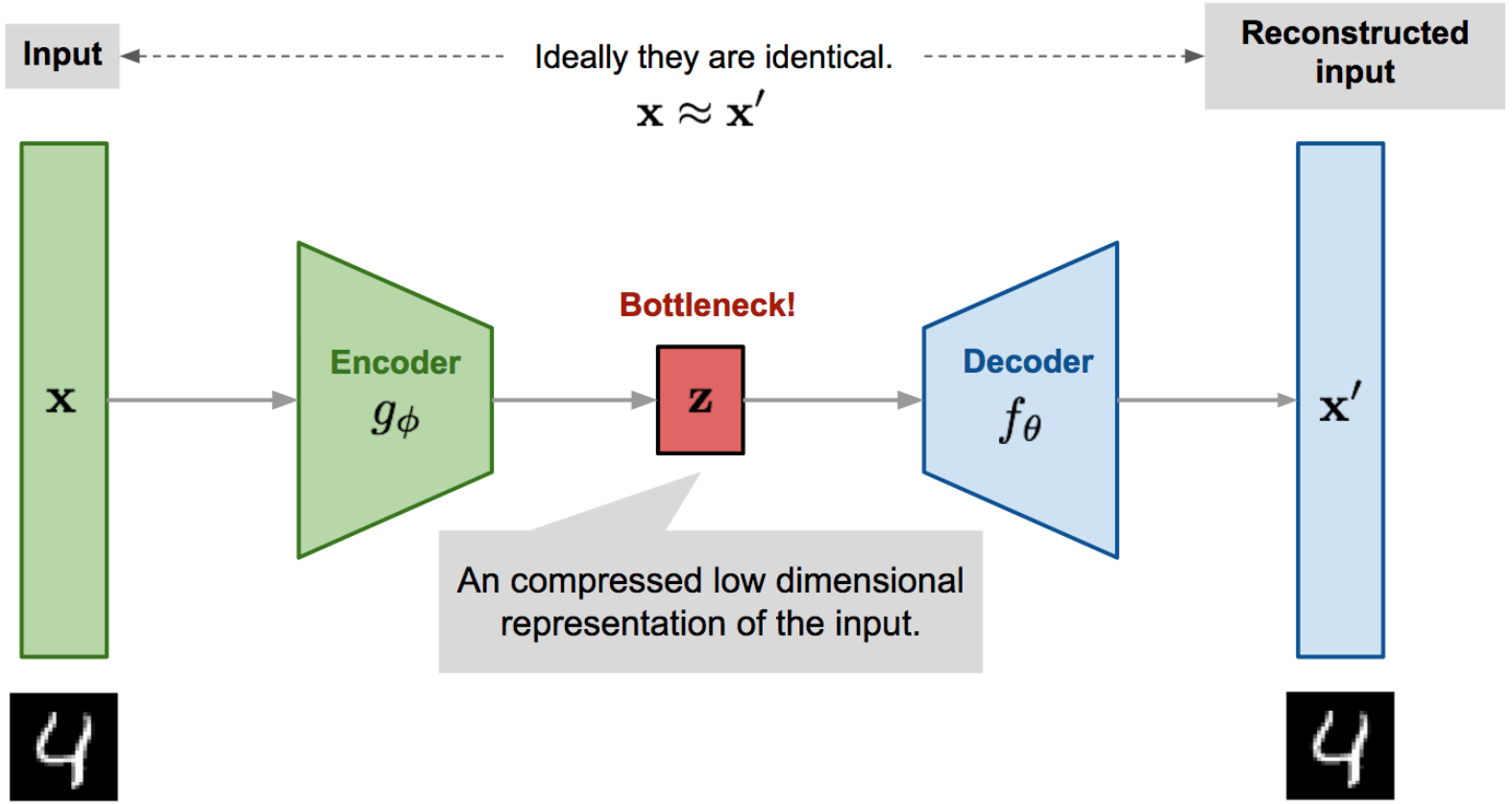
참고 자료: 데이비드 포스터, 만들면서 배우는 생성 AI (2판), 한빛미디어, 2022

AutoEncoder의 구조



AutoEncoder의 구조

- 인코더 : 이미지 같은 고차원 입력 데이터를 저차원 임베딩 벡터로 압축합니다.
- 디코더 : 임베딩 벡터를 원본 도메인으로 압축 해제합니다.



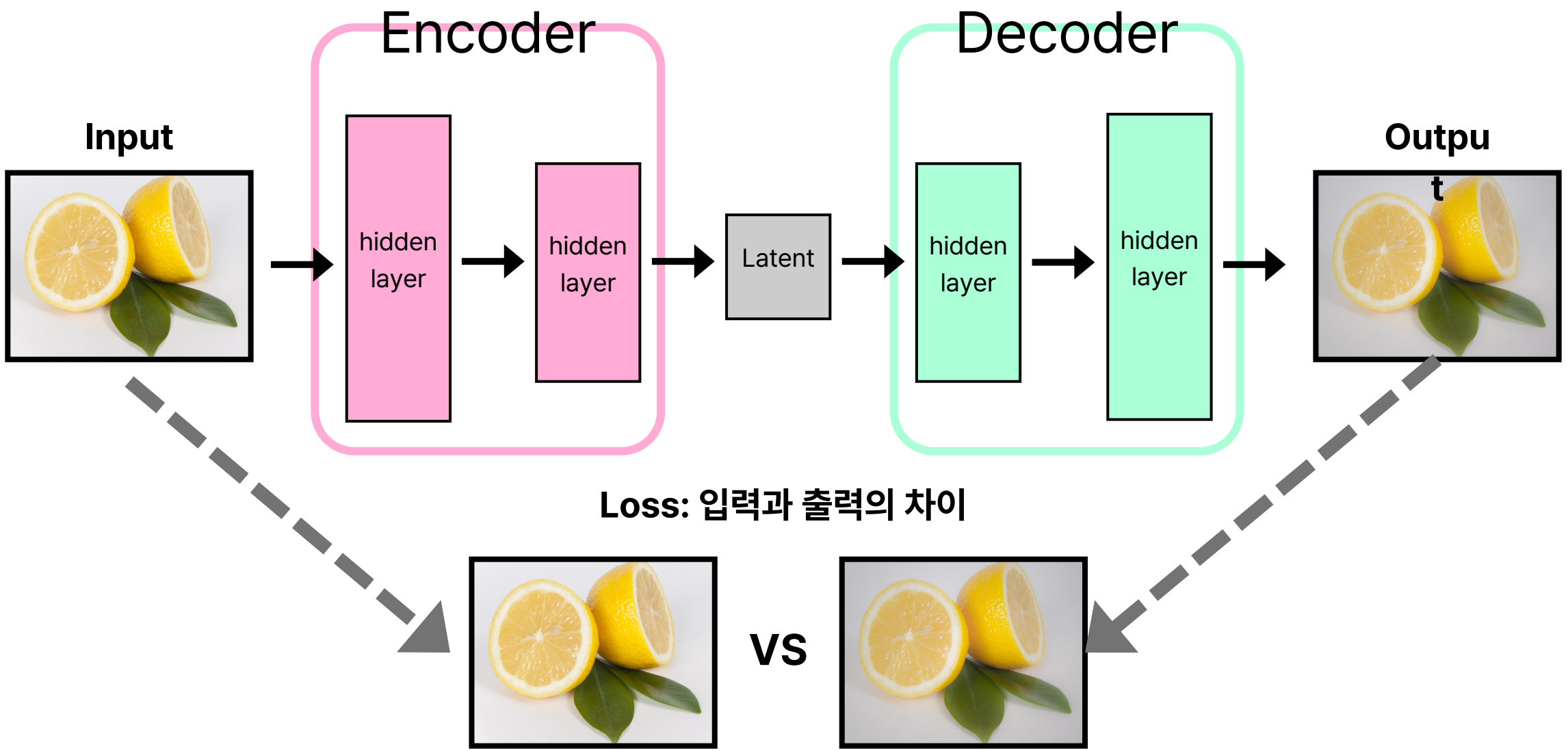
출처 : https://lilianweng.github.io/posts/2018-08-12-vae/

Encoder (인코더)

- 입력 데이터를 저차원(latent space) 표현으로 변환하는 역할
- 일반적으로 다층 퍼셉트론(MLP) 또는 CNN, RNN을 사용하여 구성
- 입력 데이터보다 작은 차원의 숨겨진 표현(잠재 변수, latent representation)을 학습

Decoder (디코더)

- 인코더가 만든 저차원 표현을 원래 데이터 차원으로 복원하는 역할
- 인코더의 반대 구조를 가짐
- 복원된 출력이 원본 입력과 최대한 유사하도록 학습


Encoder
Decoder
Input
hidden layer
hidden layer
Latent
hidden layer
hidden layer
Output
Loss: 입력과 출력의 차이
VS

AutoEncoder의 학습과정

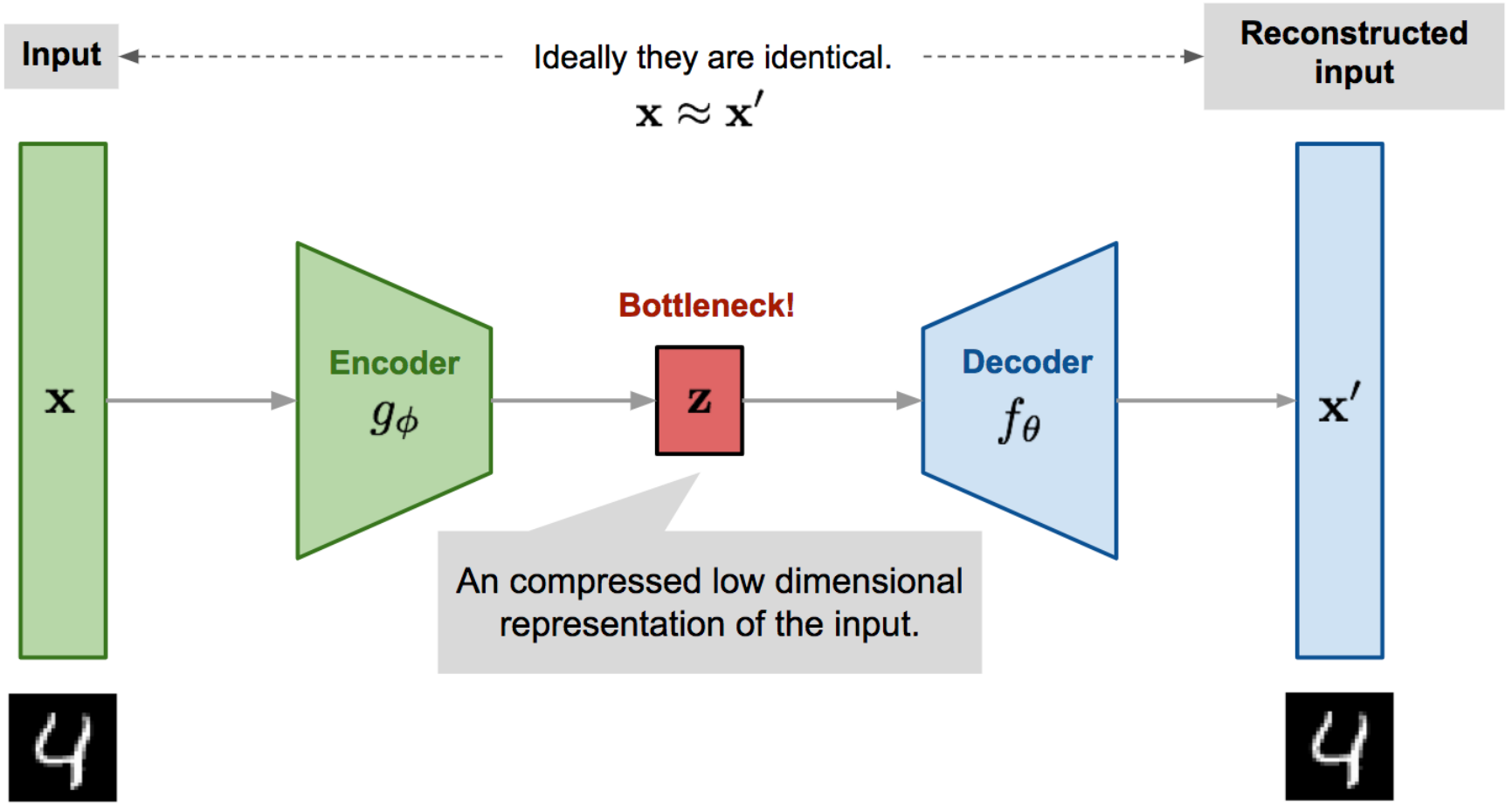
AutoEncoder는 입력과 출력을 동일하게 두고 학습하는 특징이 있습니다.

1. 입력 데이터(x)를 인코더에 전달하여 잠재 변수(z)를 생성
2. 디코더를 통해 z에서 원본과 유사한 데이터(x')를 복원
3. 입력 x와 출력 x' 간의 차이를 최소화하도록 손실 함수(예: MSE)를 사용하여 모델 최적화

$$L = ||x - x'||^2$$

라벨을 사용하지 않습니다!

AutoEncoder



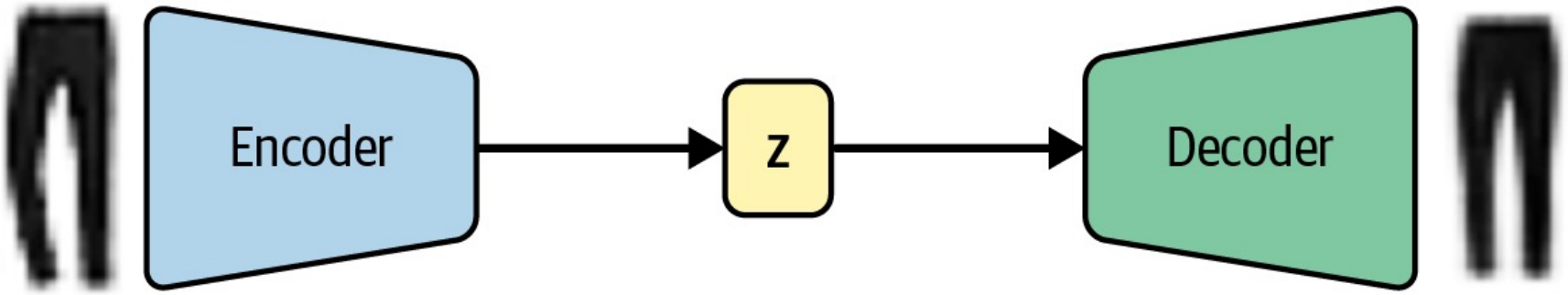
https://dataplay.tistory.com/34

오토인코더는 표현학습(representation learning)을
비지도 학습(unsupervised learning) 형태로 훈련하는 신경망

원본 이미지

잠재 임베딩 벡터

원본 픽셀 공간으로
디코딩



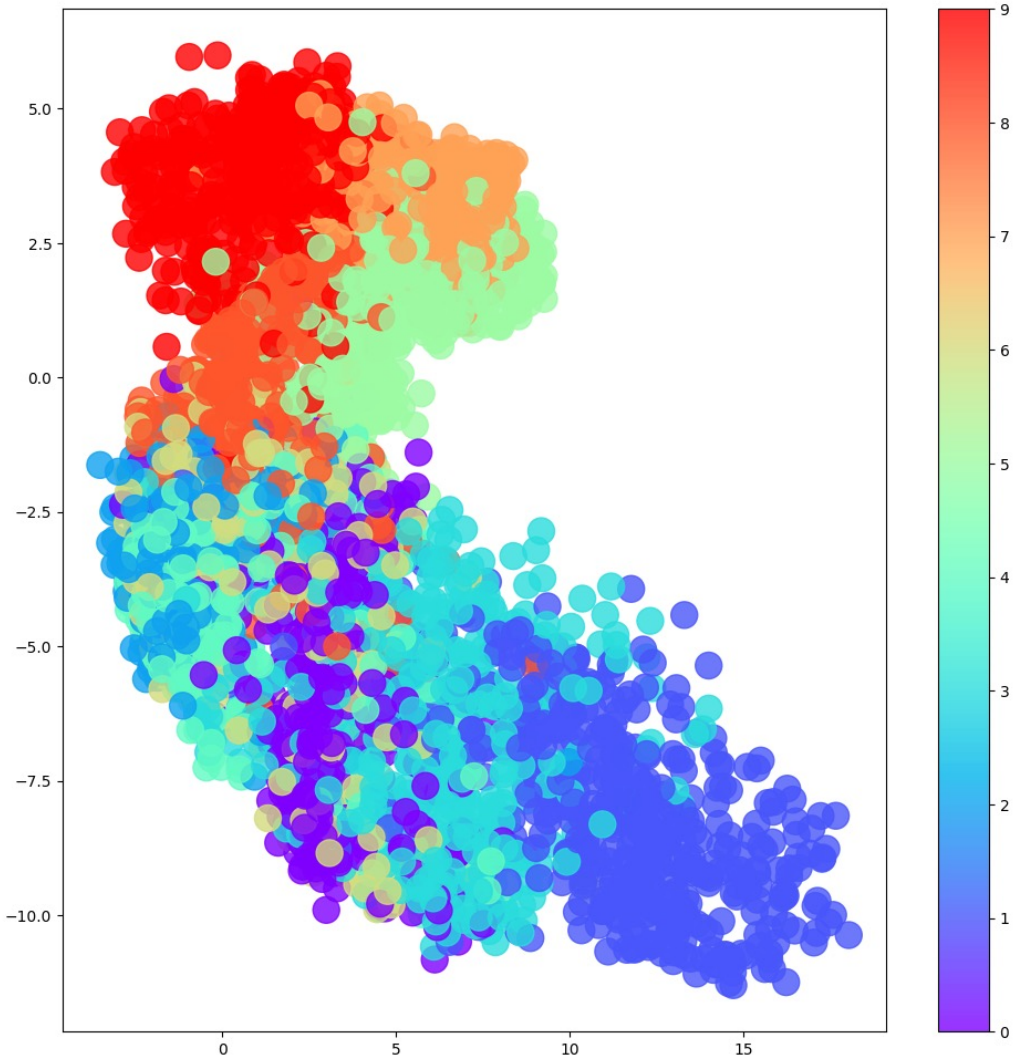
오토 인코더

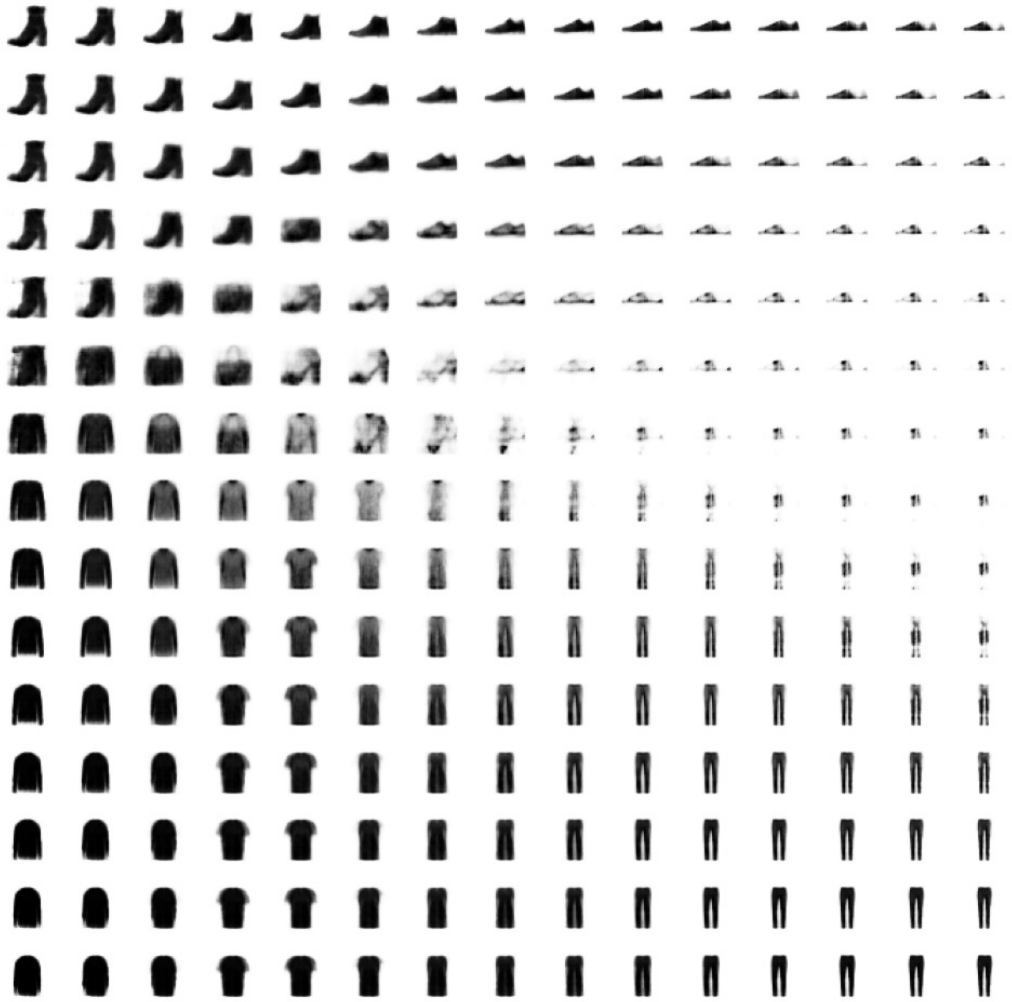
- 구조: 인코더와 디코더로 구성되어 입력 데이터를 압축한 후 다시 복원합니다.
- 목적: 데이터의 중요한 특징을 추출하여 차원 축소나 노이즈 제거 등 다양한 용도로 사용됩니다.
- 특징: 결정론적 매핑을 수행하며, 입력 데이터를 그대로 재현하는 데 초점을 맞춥니다.

오토 인코더의 생성 모델로서의 한계

- 결정론적 특성: 잠재 공간이 고정된 값으로 표현되기 때문에 새로운 데이터를 생성하는 데 한계가 있음
- 잠재 공간 규제 미흡: 잠재 변수들이 의미 있는 분포를 형성하지 않아, 임의의 샘플링 시 비현실적인 결과가 나타날 수 있음

AutoEncoder





https://github.com/rickiepark/Generative_Deep_Learning_2nd_Edition/blob/main/notebooks/03_vae/01_autoencoder/autoencoder.ipynb

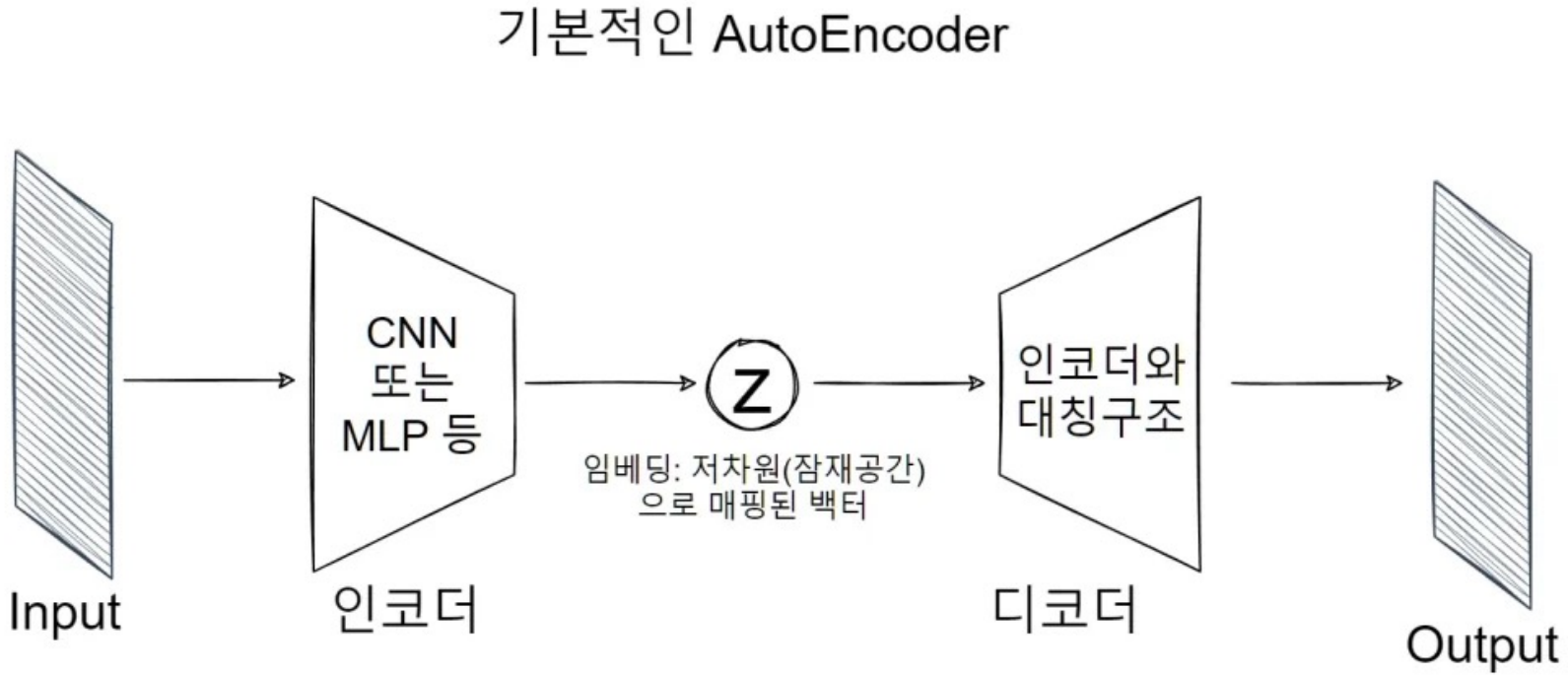
Variational AutoEncoder

- VAE는 생성 모델의 한 종류로, 입력 데이터를 저차원 잠재 공간(latent space)의 확률 분포로 인코딩한 후, 이 공간에서 샘플링하여 데이터를 재구성하는 모델입니다. (2013)
- 핵심 아이디어
  - **확률적 인코딩:** 데이터를 하나의 결정적 값이 아니라, 분포(예: 정규분포)로 표현하여 다양한 샘플을 생성할 수 있음
  - **생성 능력:** 잠재 공간에서 샘플링하여 새로운 데이터를 생성

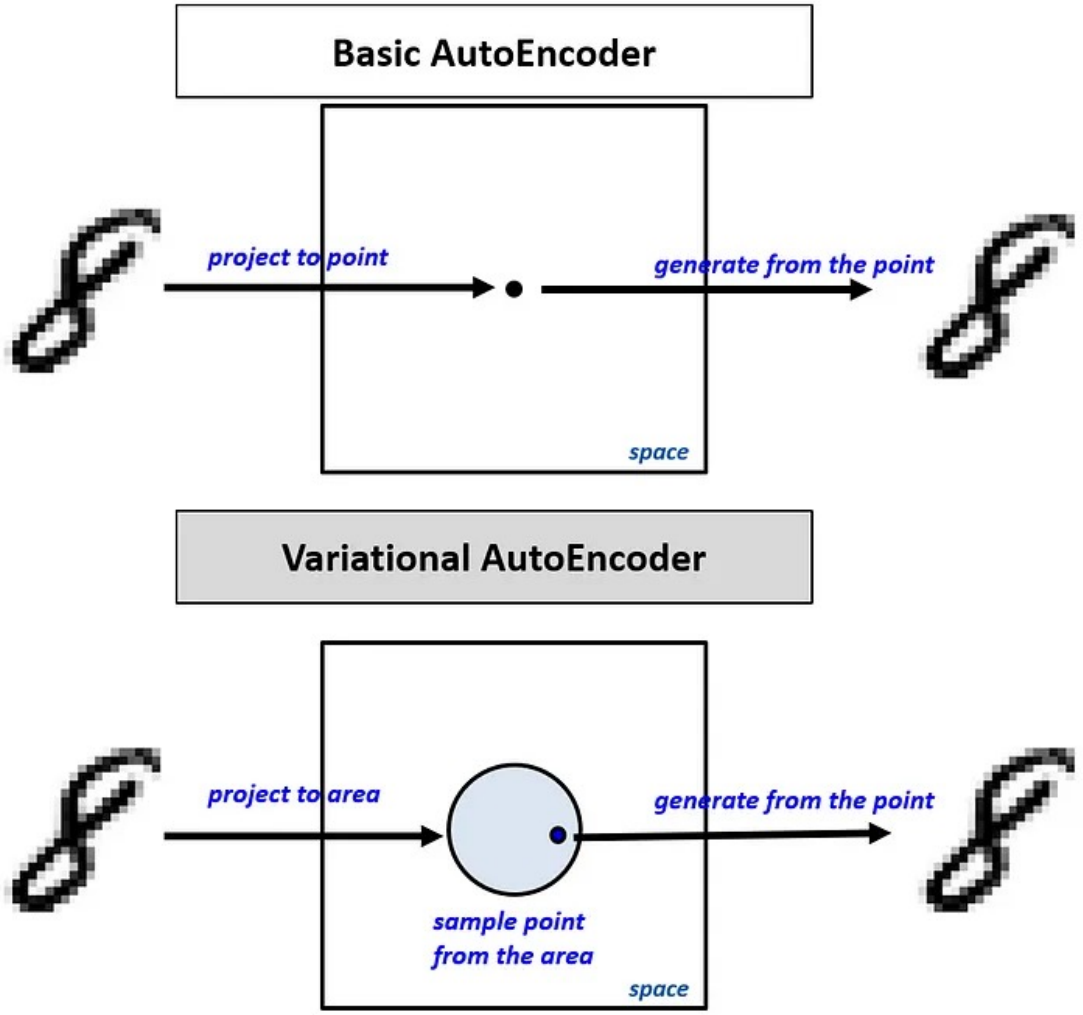
- Auto-Encoding Variational Bayes https://arxiv.org/pdf/1312.6114

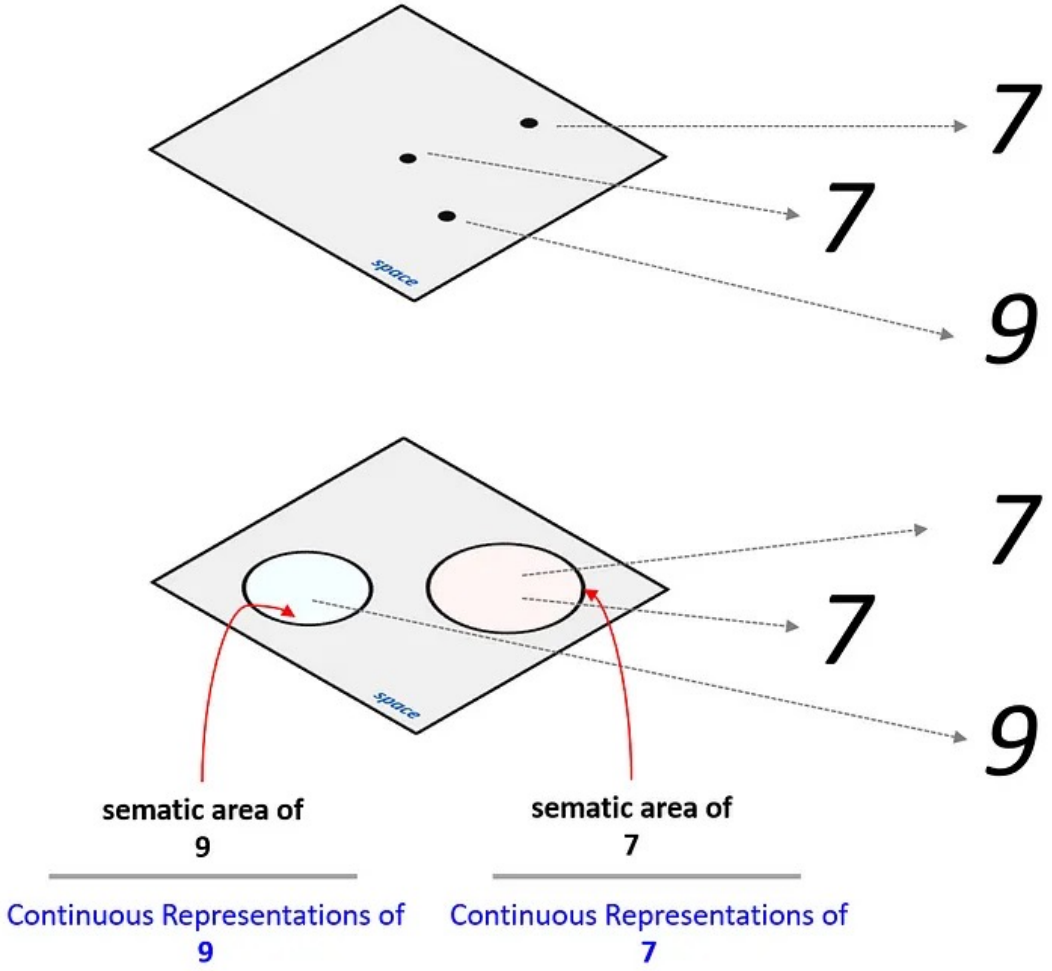
사실 오토인코더는 차원축소를 위해 탄생한 네트워크다!

생성 모델을 위해 고안된 AE의 변형이 있다. -> VAE



https://alltommysworks.com/vae/

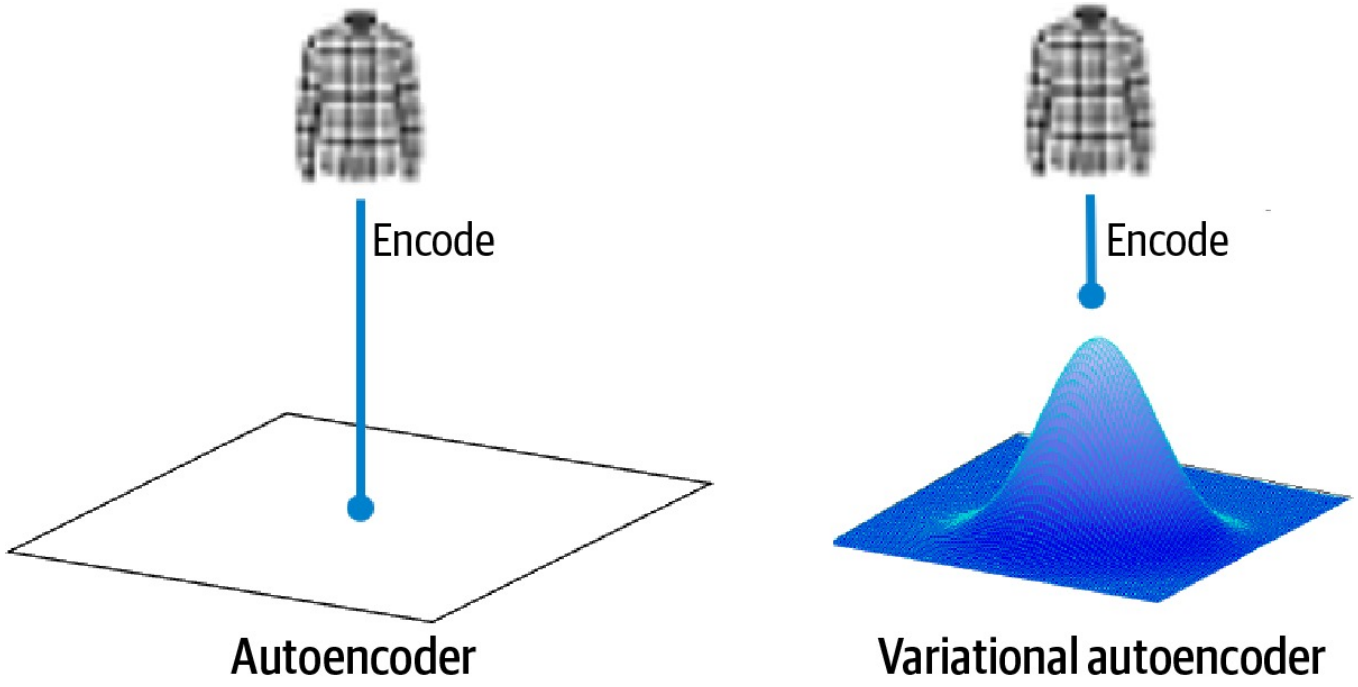




https://medium.com/@hugmanskj/autoencoder-%EC%99%80-variational-autoencoder%EC%9D%98-%EC%A7%81%EA%B4%80%EC%A0%81%EC%9D%B8-%EC%9D%B4%ED%95%B4-171b3968f20b

잠재공간 구성 : 점에서 영역으로

- AE에서는 각 이미지가 잠재 공간의 한 포인트에 직접 매핑됩니다
- VAE는 각 이미지가 잠재 공간에 있는 포인트 주변의 다변량 정규분포에 매핑됩니다.



https://medium.com/@hugmanskj/autoencoder-%EC%99%80-variational-autoencoder%EC%9D%98-%EC%A7%81%EA%B4%80%EC%A0%81%EC%9D%B8-%EC%9D%B4%ED%95%B4-171b3968f20b

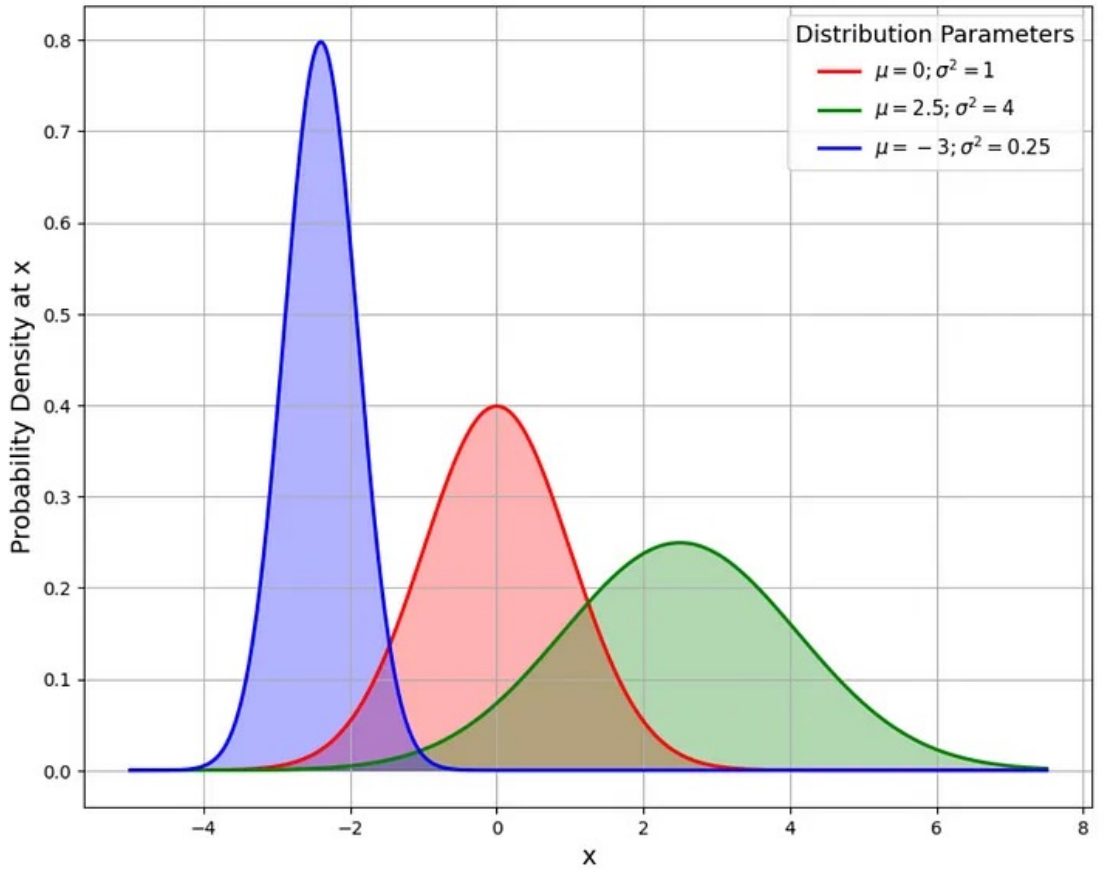
정규분포(가우시안 분포)

가우시안 분포(정규분포)는 연속 확률 분포 중 하나로, 평균값 주변에 데이터가 집중되고, 멀어질수록 확률이 점차 감소하는 '종 모양(bell curve)'의 분포를 말합니다.

주요 파라미터:

- 평균 (μ): 분포의 중심을 나타내며, 데이터의 중심 위치를 의미합니다.
- 표준편차 (σ): 데이터가 평균 주변에서 얼마나 퍼져 있는지를 나타내며, 분포의 폭을 결정합니다.

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$



https://medium.com/@hugmanskj/autoencoder-%EC%99%80-variational-autoencoder%EC%9D%98-%EC%A7%81%EA%B4%80%EC%A0%81%EC%9D%B8-%EC%9D%B4%ED%95%B4-171b3968f20b

정규분포(가우시안 분포)

다변량 정규분포 (Multivariate Normal Distribution)

- 다변량 정규분포는 여러 변수로 구성된 데이터에 대한 정규분포 확장이며, 각 변수 간의 상관관계(공분산)를 동시에 고려합니다.
- 평균 벡터 (μ): 각 변수의 평균값을 모은 벡터.
- 공분산 행렬 (Σ): 각 변수의 분산과 변수 간의 공분산을 나타내며, 잠재 공간 내 변수들 사이의 관계를 규정합니다.

$$f(\mathbf{x}) = \frac{1}{\sqrt{(2\pi)^k|\Sigma|}} \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\top \Sigma^{-1}(\mathbf{x}-\boldsymbol{\mu})\right)$$

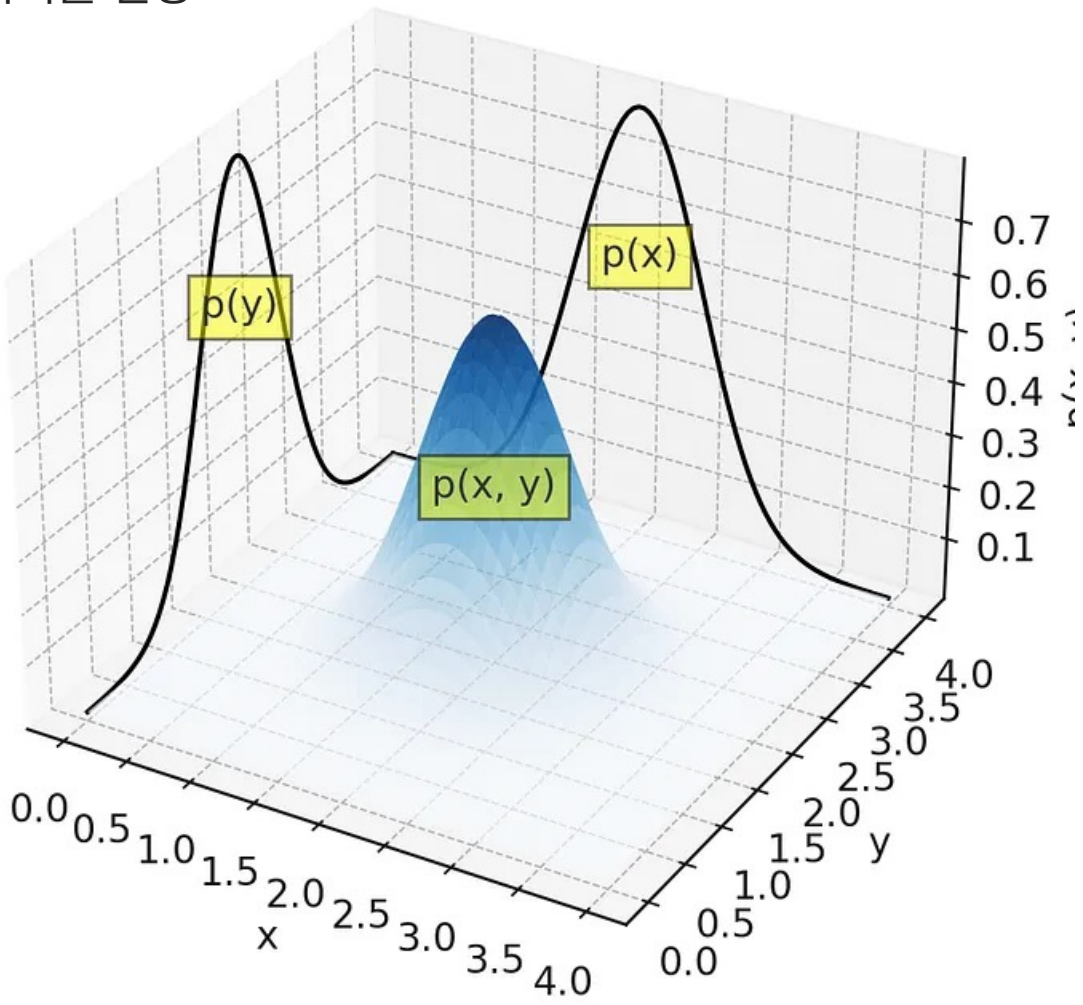
- 다차원 잠재 공간:
  VAE는 일반적으로 다차원의 잠재 공간을 사용합니다. 이 경우, 잠재 변수 z는 다변량 정규분포를 따르도록 모델링됩니다.
- 구현상의 단순화:
  많은 VAE 구현에서는 공분산 행렬을 대각행렬로 가정합니다.
  즉, 변수들 간의 상관관계는 0으로 두고 각 변수마다 독립적인 분산을 사용합니다.
  이 경우, 다변량 정규분포는 각 차원별로 독립된 가우시안 분포의 곱으로 표현할 수 있습니다

정규분포(가우시안 분포)

2차원 상에 가우시안 분포로 '영역'을 그리려면 총 4개의 파라미터를 설정

(x, y 독립 가정)

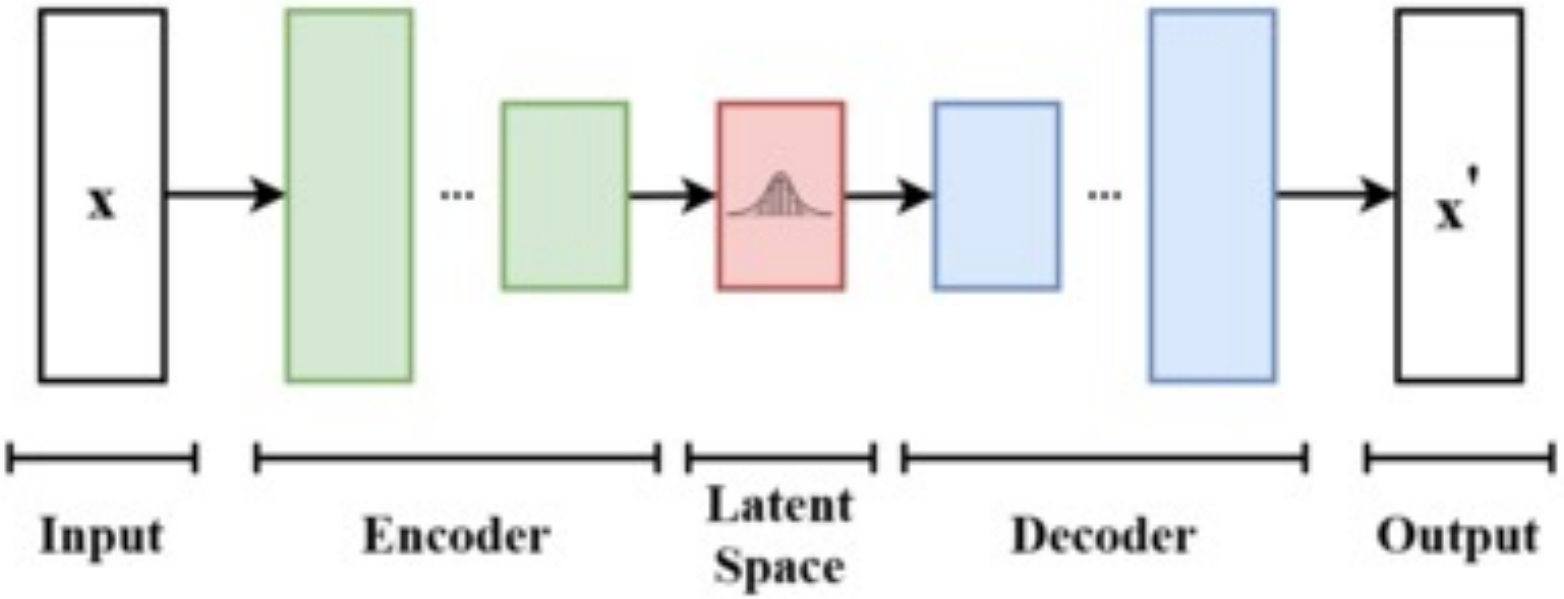
- μ_x: x축에서의 평균값
- μ_y: y축에서의 평균값
- σ_x: x축에서의 표준편차
- σ_y: y축에서의 표준편차



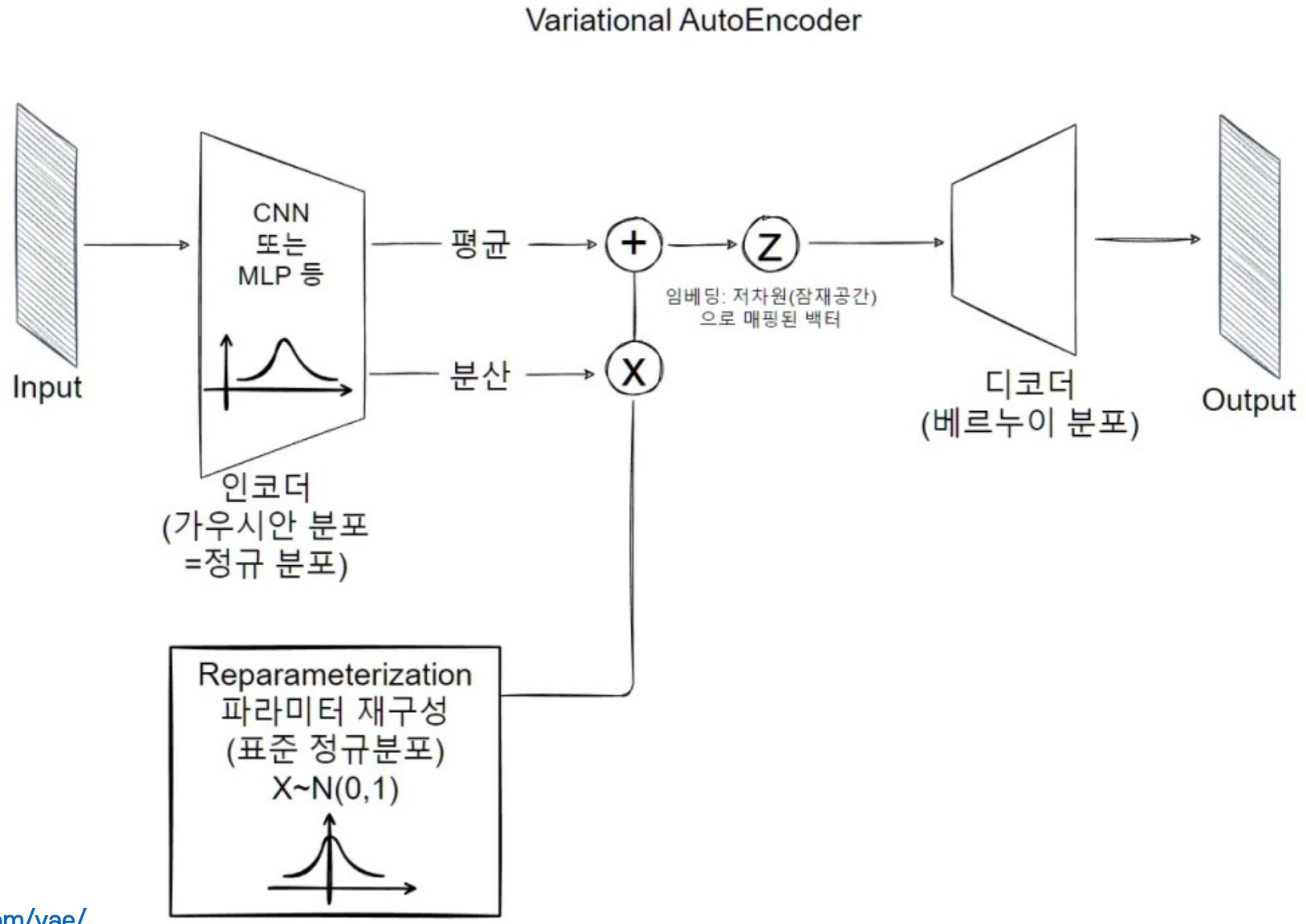
https://medium.com/@hugmanskj/autoencoder-%EC%99%80-variational-autoencoder%EC%9D%98-%EC%A7%81%EA%B4%80%EC%A0%81%EC%9D%B8-%EC%9D%B4%ED%95%B4-171b3968f20b

VAE Architecture

- 인코더가 입력 x를 받아서 잠재 공간으로 압축하고. 디코더가 잠재공간에서 결과를 받아서 입력과 가까운 x'로 복원한다.
- **디코더의 입력은 무엇인가요?**
- **어떻게 이 모델은 역전파가 가능할까요?**



https://ko.wikipedia.org/wiki/%EB%B3%80%EB%B6%84_%EC%98%A4%ED%86%A0%EC%9D%B8%EC%BD%94%EB%8D%94



https://alltommysworks.com/vae/

VAE 구조

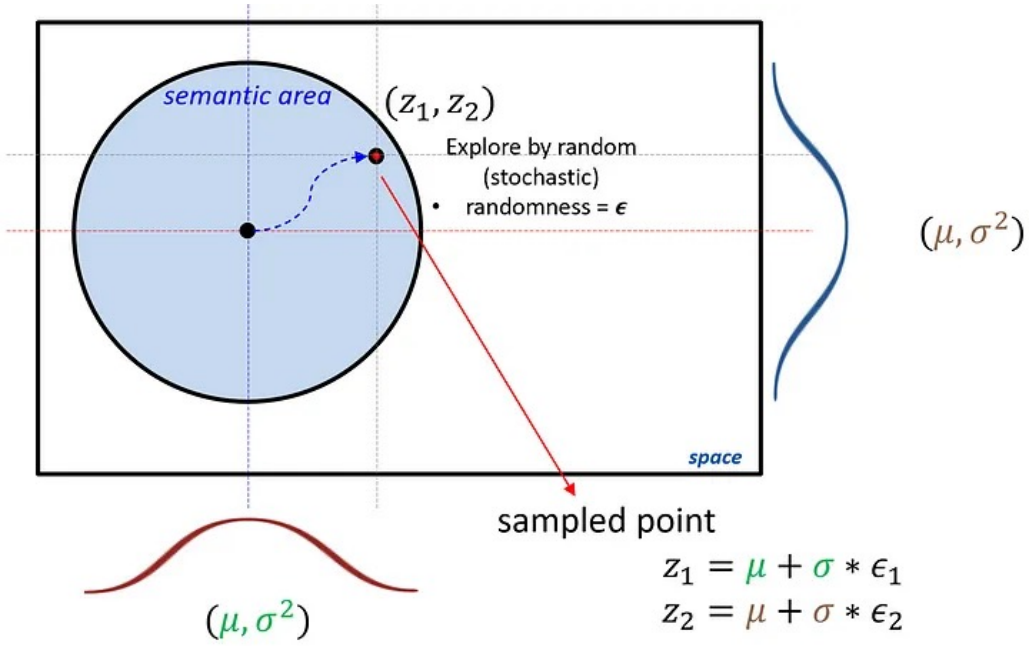
- 인코더: 입력 x로부터 μ(x))와 σ(x)를 예측해, z가 정규분포(N(μ,σ2))를 따르도록 함
- 디코더: 샘플링된 z를 입력받아, 원본 데이터를 재구성하거나 새로운 데이터를 생성

샘플링

- 인코더가 μ=0.5, σ=0.2라고 예측했다면,
  실제로는 z~N(0.5,0.22)에서 무작위로 샘플링

샘플링 과정의 문제점 : 역전파 불가

- 딥러닝에서 모델을 학습하기 위해서는 역전파를 통해 가중치를 업데이트해야 합니다. 그런데 일반적인 "무작위 샘플링" 연산은 미분이 불가능합니다.
- sample(N(μ,σ2)) 같은 연산은 내부적으로 난수 생성이 들어가므로, μ나 σ에 대해 미분(gradient)을 계산하기 어렵습니다.



https://medium.com/@hugmanskj/autoencoder-%EC%99%80-variational-autoencoder%EC%9D%98-%EC%A7%81%EA%B4%80%EC%A0%81%EC%9D%B8-%EC%9D%B4%ED%95%B4-171b3968f20b

재파라미터화 트릭(Reparameterization Trick)

샘플링을 직접 N(μ,σ2) 에서 하지 않고, 표준 정규분포(N(0,1) 에서 샘플링한 뒤에 μ와 σ를 이용해 변환(transform)하는 방식으로 바꿉니다.

$$\epsilon \sim \mathcal{N}(0,1) \quad \longrightarrow \quad z = \mu + \sigma \cdot \epsilon$$

- ϵ은 평균 0, 표준편차 1인 표준 정규분포에서 샘플링한 무작위 변수
- z=μ+σ·ϵ을 통해 원하는 정규분포 N(μ,σ2)로 변환

ϵ은 샘플링 과정에서 고정된 "난수 입력"으로 보고,
μ와 σ가 어떻게 바뀌어야 재구성 오류와 KL Divergence 등이 줄어드는지 계산할 수 있게 됩니다.

Log Variance의 사용

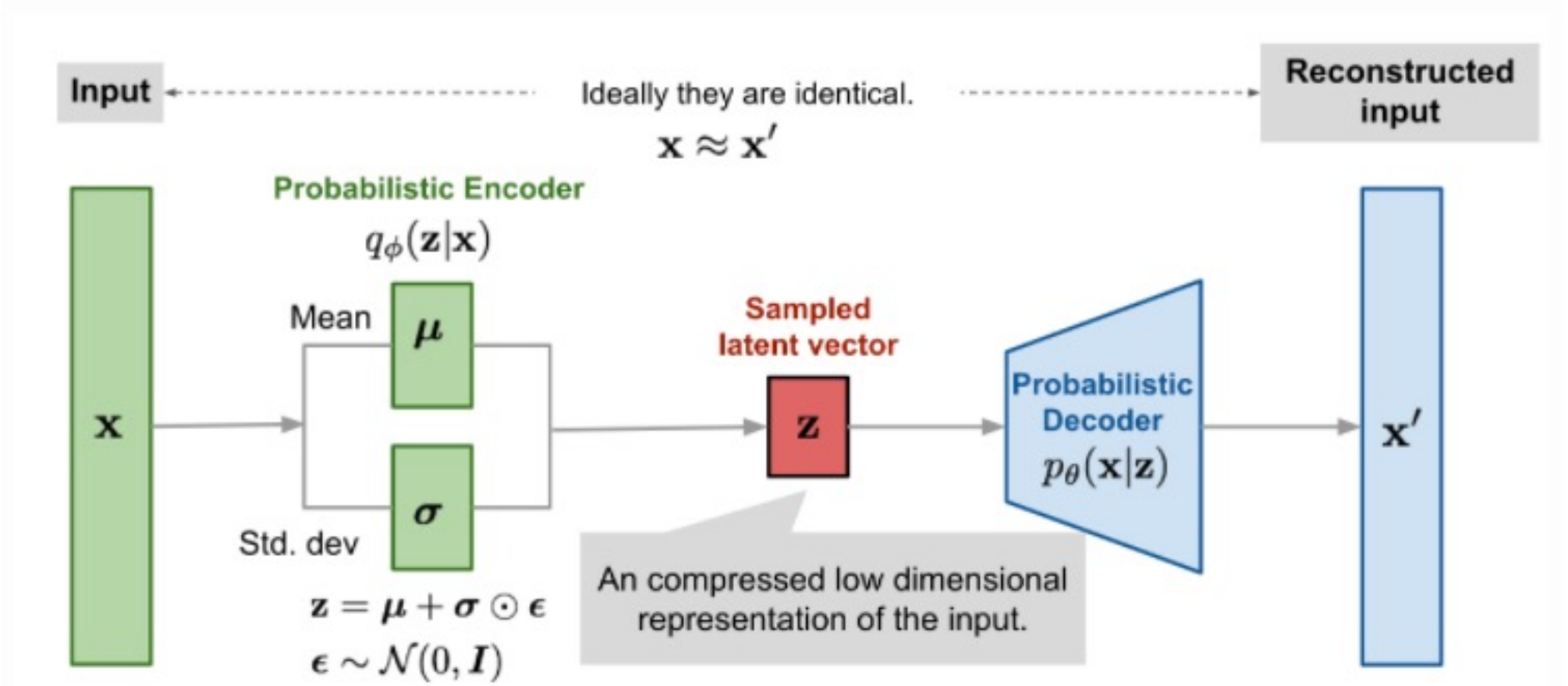
이때, 모델은 μ와 σ를 직접 예측하기도 하지만, σ대신 로그 분산(log σ^2)이나 로그 표준편차(log σ) 등을 예측하는 방식을 자주 사용합니다.

$$\text{Encoder Output: } (\mu, \log \sigma^2) \quad \longrightarrow \quad \sigma^2 = \exp(\log \sigma^2)$$

Log Variance을 사용하는 이유

- 수치적 안정성(Numerical Stability)
  - 분산은 항상 양수여야 합니다.모델이 분산을 음수로 예측하면 안 되므로, 학습 시에 제약을 추가해야 합니다.
  - 로그 스케일로 예측하면,어떤 실수 값을 출력해도 exp (·)를 취하면 양수가 되므로, 자동으로 양수 제약을 만족합니다.
  - 분산이 0 근처이거나 매우 큰 값이 되는 극단적인 상황에서도 문제가 덜 발생합니다.

VAE



https://gaussian37.github.io/dl-concept-vae/

LOSS : 재구성 손실(Reconstruction Loss) + 잠재 공간 정규화 손실(Latent Space Regularization Loss)

$$\text{Loss} = \text{Reconstruction Loss} + \beta \cdot D_{KL}(q(z|x) \| p(z))$$

재구성 손실 (Reconstruction Loss)

- 디코더가 인코더로부터 전달받은 잠재 변수 z로부터 원본 입력 x를 얼마나 잘 재구성하는지를 평가합니다.
- 입력 x와 디코더가 재구성한 $\hat{x}$사이의 차이를 측정하는데, 대표적으로 평균 제곱 오차(MSE)나 이진 교차 엔트로피 (BCE) 같은 지표가 사용됩니다

잠재 공간 정규화 손실 (Latent Space Regularization Loss)

- 인코더가 생성한 잠재 변수 분포 q(z|x)와 사전에 가정한 정규분포 p(z) (보통 표준 정규분포 N(0,I)) 사이의 차이를 줄이는 역할을 합니다
- KL Divergence (Kullback-Leibler Divergence) 를 통해 측정되며, 다음과 같이 수식으로 표현됩니다

$$D_{KL}\left(q(z|x) \| p(z)\right) = \mathbb{E}_{q(z|x)}\left[\log \frac{q(z|x)}{p(z)}\right]$$

KL Divergence

KL Divergence는 두 확률 분포 P와 Q가 얼마나 다른지,
즉 Q로 P를 근사할 때 얼마나 많은 추가 정보가 필요한지를 수식으로 나타낸 것입니다

$$D_{KL}(P\|Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)} \qquad D_{KL}(P\|Q) = \int P(x) \log \frac{P(x)}{Q(x)} dx$$

- $P(x)$는 실제 분포,
- $Q(x)$는 근사하려는 예측 분포입니다.

크로스 엔트로피와의 관계

KL Divergence는 교차 엔트로피에서 P의 엔트로피를 뺀 나머지, 즉 Q가 P에 비해 얼마나 비효율적인지를 나타냅니다.

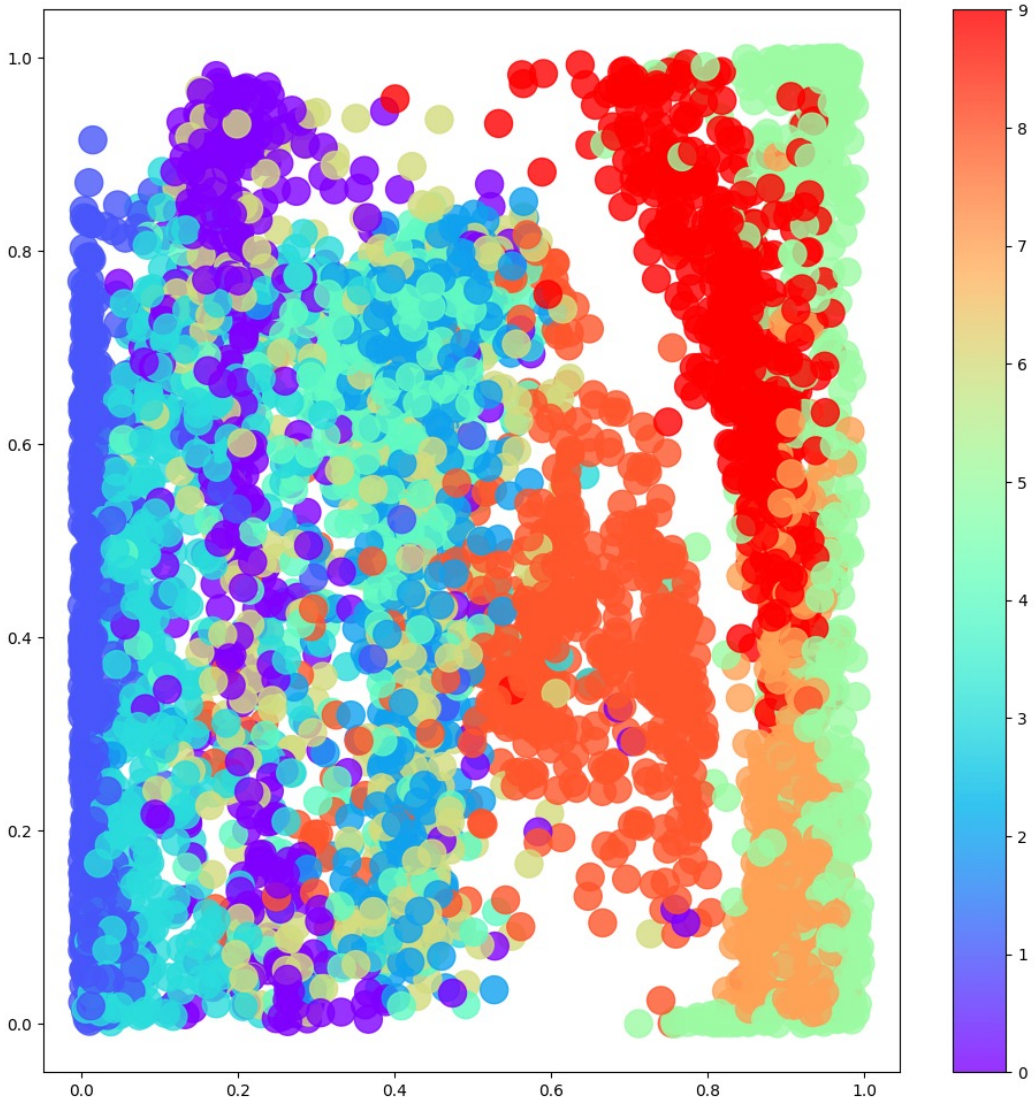
$$H(P,Q) = H(P) + D_{KL}(P\|Q)$$

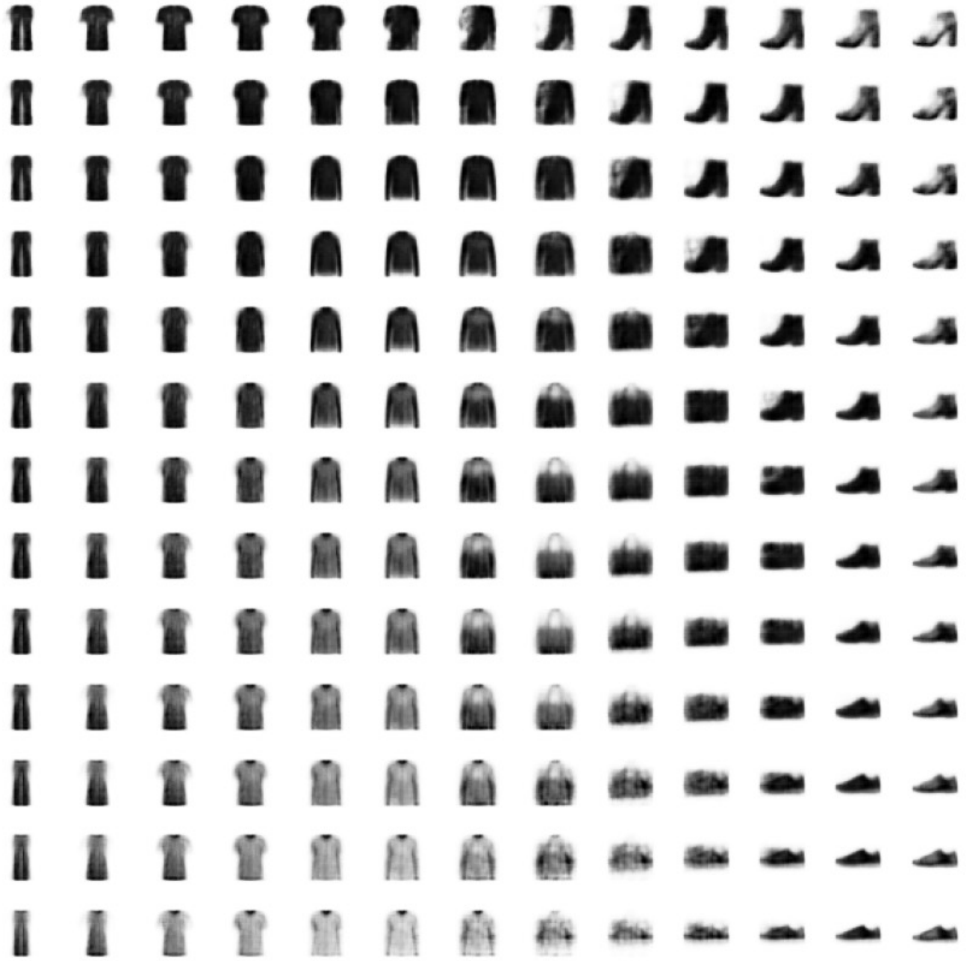
크로스 엔트로피와의 관계

KL Divergence는 교차 엔트로피에서 P의 엔트로피를 뺀 나머지, 즉 Q가 P에 비해 얼마나 비효율적인지를 나타냅니다.

$$
\begin{aligned}
D_{KL}(P \parallel Q) &= \sum P(x) \log \left(\frac{P(x)}{Q(x)} \right) \\
&= \sum P(x)(\log P(x) - \log Q(x)) \\
&= -\sum P(x)(\log Q(x) - \log P(x)) \\
&= \underbrace{-\sum P(x) \log Q(x)}_{\text{Cross Entropy}} - \underbrace{(-\sum P(x) \log P(x))}_{\text{Entropy}}
\end{aligned}
$$

VAE





https://github.com/rickiepark/Generative_Deep_Learning_2nd_Edition/blob/main/notebooks/03_vae/02_vae_fashion/vae_fashion.ipynb

VAE의 장점

- 연속적이고 해석 가능한 잠재 공간: 확률 분포를 이용하여 의미 있는 영역(semantic area)을 형성하므로,
임의 샘플링 시에도 자연스러운 데이터 생성을 가능하게 합니다.
- 생성 능력: 학습된 잠재 공간에서 새로운 데이터를 생성할 수 있으며, 데이터 보강이나 변형에 유용합니다.
- 학습 안정성: 재파라미터화 트릭과 KL Divergence를 통한 규제 덕분에, Autoencoder보다 안정적인 학습이 가능합니다.

VAE의 한계

- 재구성 품질: Autoencoder에 비해 재구성 결과가 다소 흐릿하거나 덜 세밀할 수 있음.
- 하이퍼파라미터 튜닝: 재구성 손실과 KL Divergence 사이의 균형을 맞추기 위해 β등 하이퍼파라미터 조절이 필요함.
- 잠재 분포의 제약: 사전 분포에 과도하게 의존할 경우, 데이터의 복잡한 특성을 온전히 반영하지 못할 수 있음.

03

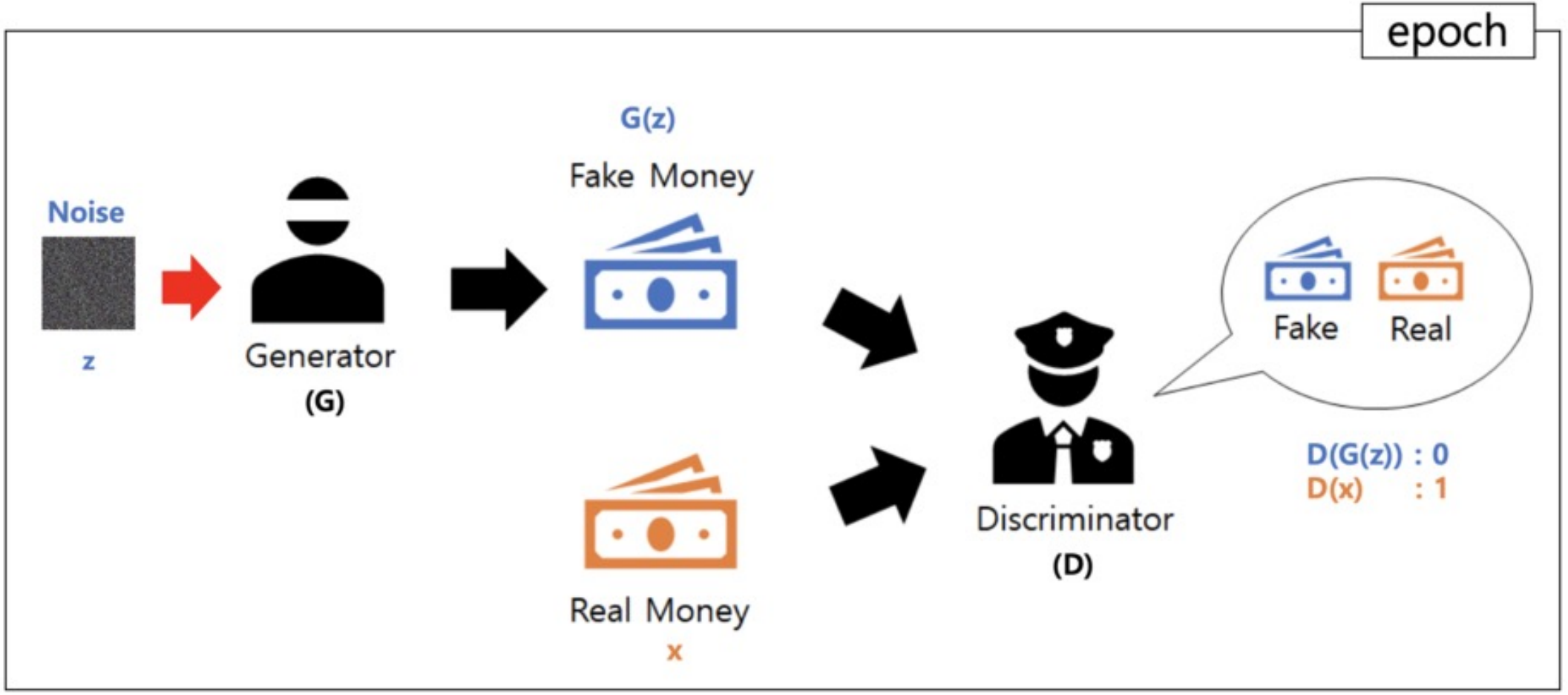
Generative Adversarial Network

Generative Adversarial Network

GAN은 생성 모델 중 하나로, 두 개의 신경망—생성자(Generator)와 판별자(Discriminator)—가 서로 경쟁하며 학습하는 방식입니다. (2014)

핵심 아이디어

- 생성자(Generator): 무작위 노이즈로부터 현실적인 데이터를 생성하려고 시도합니다.
- 판별자(Discriminator): 입력된 데이터가 실제(real) 데이터인지 생성된(fake) 데이터인지를 구분하려고 합니다.
- Generative Adversarial Networks https://arxiv.org/pdf/1406.2661



https://baechu-story.tistory.com/12

구성 요소

- **생성자(Generator) G:**
  - 입력: 랜덤 노이즈 z (예: 표준 정규분포로부터 샘플링)
  - 출력: 실제 데이터와 유사한 "가짜" 데이터 $\hat{x}$=G(z)
- **판별자(Discriminator) D:**
  - 입력: 실제 데이터 x 또는 생성된 데이터 G(z)
  - 출력: 데이터가 실제일 확률 D(x) (보통 0에서 1 사이의 값)

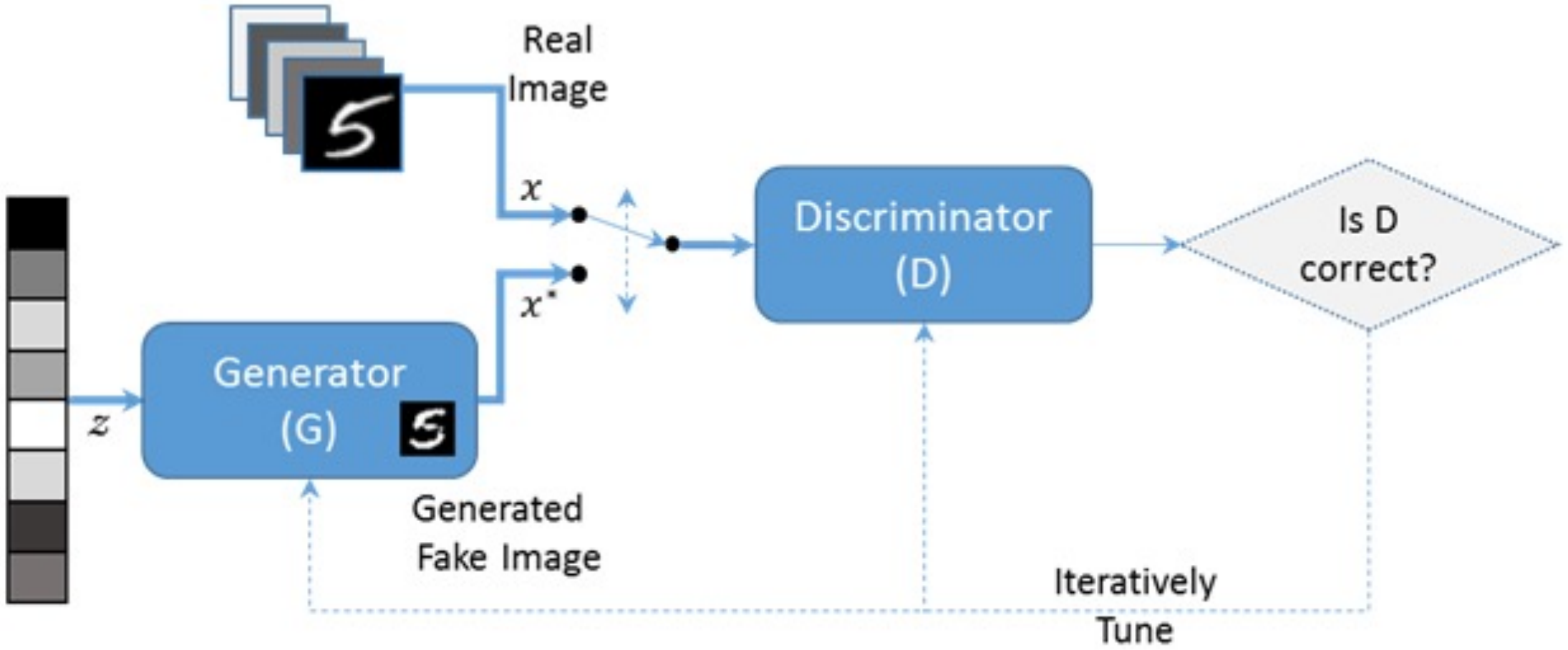
적대적 신경망(Adversarial Nets)

생성자는 판별자를 속여 가짜 데이터를 진짜처럼 보이게 만들고, 판별자는 이를 분별하려고 합니다.

•목표:

- **생성자:** 판별자가 생성된 데이터를 진짜라고 오판하도록 만드는 것
- **판별자:** 실제 데이터와 생성 데이터를 올바르게 구분하는 것

GAN



https://itwiki.kr/w/GAN

Objective Function

GAN은 미니맥스 게임(minimax game) 형태의 손실 함수를 설정합니다.

$$\min_G \max_D \; V(D,G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} \left[\log D(x)\right] \; + \; \mathbb{E}_{z \sim p(z)} \left[\log\left(1 - D(G(z))\right)\right]$$

- 판별자(D)는 이 식을 극대화(maximize)하려고 합니다. (진짜는 1로, 가짜는 0으로 잘 분류)
- 생성자(G)는 이 식을 최소화(minimize)하려고 합니다. (가짜를 1로 판정받도록)

실제 학습에서 사용하는 손실 함수 형태 : 판별자(D) 손실

$$L_D = -\left(\mathbb{E}_{x\sim p_{\text{data}}(x)}[\log D(x)] + \mathbb{E}_{z\sim p(z)}[\log(1 - D(G(z)))]\right)$$

- 실제 샘플 x에 대해서는 D(x)를 크게(1에 가깝게)
- 생성 샘플 $\hat{x}$=G(z)에 대해서는 D($\hat{x}$)를 작게(0에 가깝게) 만들도록 유도

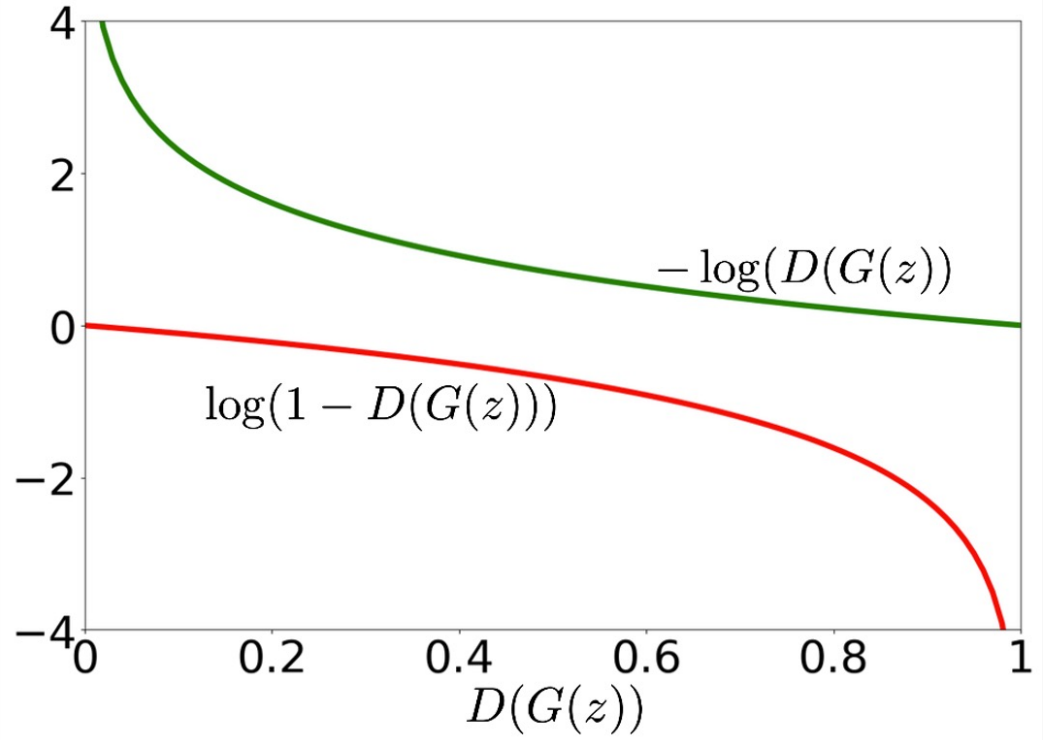
실제 학습에서 사용하는 손실 함수 형태 : 생성자(G) 손실

$$L_G = -\mathbb{E}_{z\sim p(z)}\left[\log D(G(z))\right]$$

실무에서 많이 사용

$$L'_G = \mathbb{E}_{z\sim p(z)}\left[\log(1 - D(G(z)))\right]$$

원 논문 식



https://mlbook.dev/DeepLearning/GAN.html

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k, is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

  for k steps **do**

  - Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
  - Sample minibatch of m examples $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ from data generating distribution $p_{\text{data}}(\boldsymbol{x})$.
  - Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^{m} \left[\log D\left(\boldsymbol{x}^{(i)}\right) + \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right) \right].$$

  end for

  - Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
  - Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^{m} \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right).$$

end for

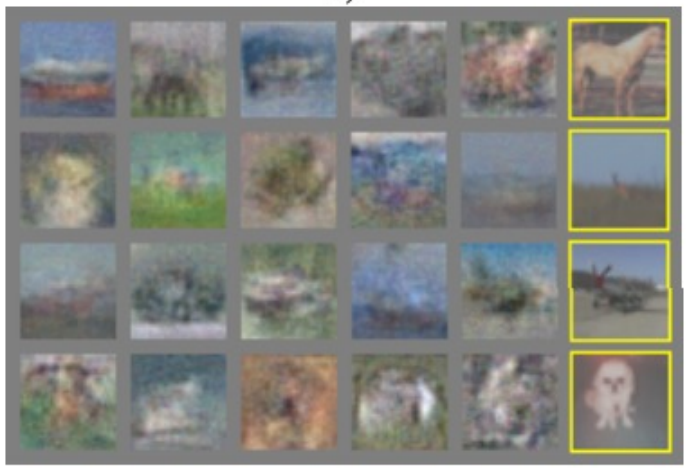
The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.



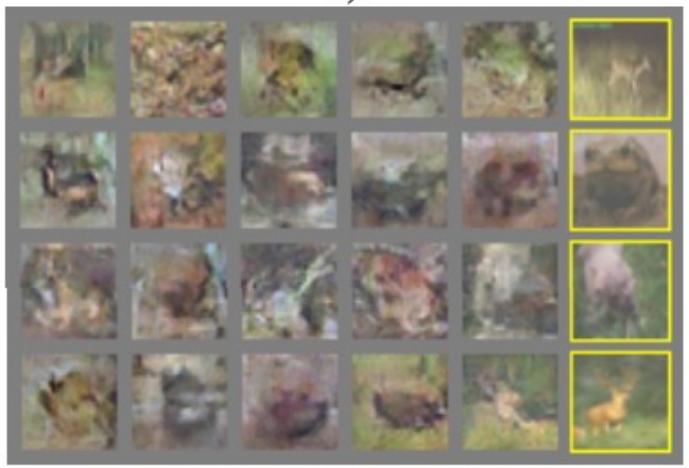
a)



b)



c)



d)





GAN에서 자주 발생하는 이슈

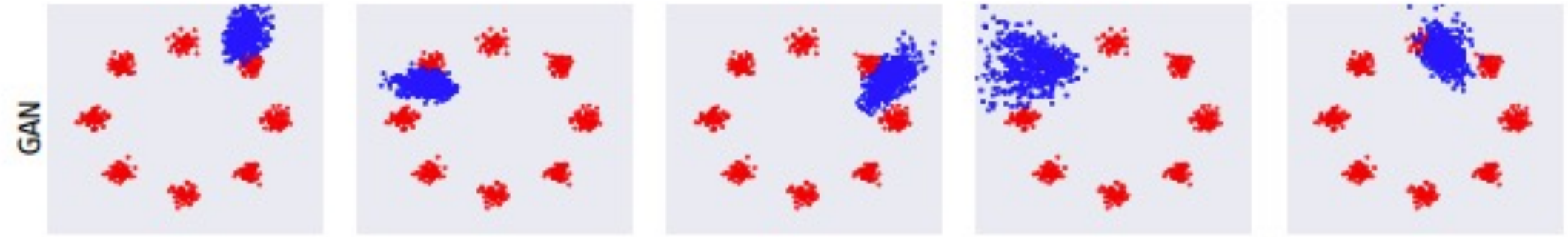
- **학습 불안정성**
  - 두 네트워크가 서로 다른 속도로 학습하거나, 한 쪽이 압도적으로 강해지면 학습이 망가질 수 있음
  - 적절한 초기화, Batch Normalization, Learning Rate 스케줄링, Gradient Penalty(WGAN-GP) 등

- **모드 붕괴(Mode Collapse)**
  - 생성자가 특정 패턴의 가짜만 만들어서 판별자를 속이는 경우
  - 예: CIFAR-10에서 "개" 이미지만 무한 반복 생성
  - 다양한 Noise Input 사용, Unrolled GAN, Mini-batch Discrimination 등 기법

- **손실값 모니터링의 어려움**
  - GAN의 손실 함수를 모니터링하기 어려워 학습 과정을 추적하는 데 어려움이 있습니다.
  - 주관적 평가:시각적으로 "괜찮아 보이는지"를 확인 → 연구자가 직접 판단
  - 완벽한 지표는 아니며, 특정 상황에서 왜곡이 있을 수 있음

GAN에서 자주 발생하는 이슈

- **모드 붕괴(Mode Collapse)**
  - 생성자가 특정 패턴의 가짜만 만들어서 판별자를 속이는 경우
  - 예: CIFAR-10에서 “개” 이미지만 무한 반복 생성



Deep Convolutional GAN

DCGAN은 Deep Convolutional Generative Adversarial Networks의 약자로, 생성적 적대 신경망(GAN)의 한 변형입니다. 이 모델은 기존 GAN의 학습 불안정 문제를 개선하기 위해, CNN 기반의 구조와 몇 가지 중요한 아키텍처 제약을 도입했습니다.

핵심 아이디어

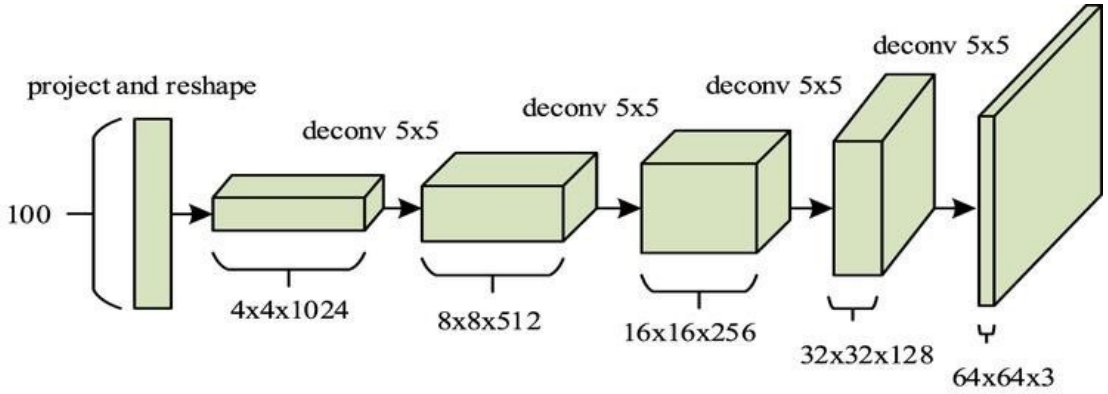
- 주요 목표 : 라벨이 없는 데이터(비지도 학습)로부터 좋은 이미지 표현(feature representation)을 학습하고, 이를 바탕으로 자연스러운 이미지를 생성하거나 분류 등 다양한 응용에 활용하는 것입니다.
- CNN의 구조적 강점을 살리면서, 풀링 대신 스트라이드 컨볼루션과 배치 정규화(Batch Normalization)를 통해 안정적인 학습을 도모합니다.
- UNSUPERVISED REPRESENTATION LEARNING WITH DEEP CONVOLUTIONAL GENERATIVE ADVERSARIAL NETWORKS
  https://arxiv.org/pdf/1511.06434

안정적인 학습을 위한 아키텍처 제약

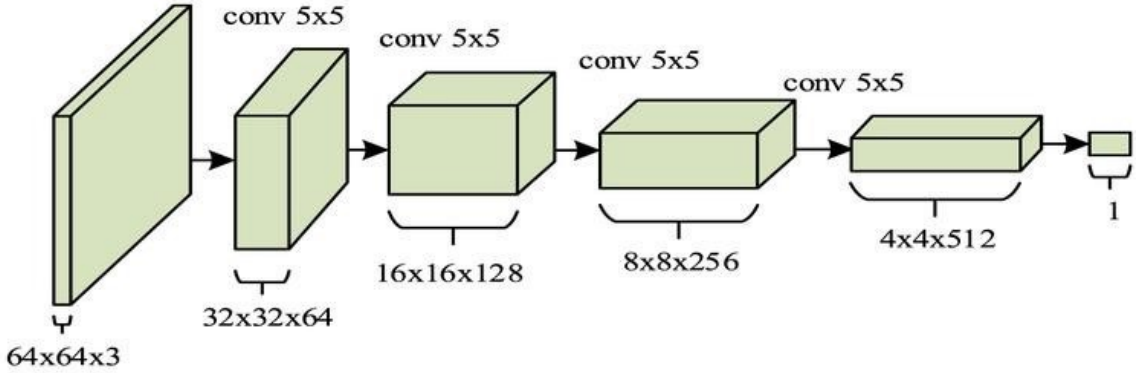
DCGAN은 GAN의 학습이 불안정했던 문제를 해결하기 위해 아래와 같은 제약 조건을 도입했습니다.

- **스트라이드 컨볼루션 사용:**
  풀링(pooling) 층 대신, 이미지의 다운샘플링(판별자)과 업샘플링(생성자)에 스트라이드 컨볼루션을 사용합니다.
- **배치 정규화(Batch Normalization):**
  각 층의 입력을 정규화하여 학습 과정에서 발생하는 내부 공변량 변화(internal covariate shift)를 줄이고, 빠르고 안정적인 수렴을 돕습니다. 단, 생성자의 출력층과 판별자의 입력층에서는 적용하지 않습니다.
- **완전 연결층 제거:**
  깊은 네트워크 구조에서 불필요한 완전 연결층을 제거하여 파라미터 수를 줄이고, 공간 정보를 보존합니다.
- **활성화 함수 선택:**
  - 생성자: 대부분의 층에서 ReLU 활성화 함수를 사용하고, 최종 출력층에서는 Tanh 함수를 사용하여 이미지 픽셀 값의 범위를 [-1, 1]로 제한합니다.
  - 판별자: LeakyReLU 활성화 함수를 사용하여, 음의 기울기도 일부 반영할 수 있게 합니다.

DCGAN



(a) Generator



(b) Discriminator

https://idiotdeveloper.com/what-is-deep-convolutional-generative-adversarial-networks-dcgan/

생성자(Generator) G:

- 입력: 잠재 벡터 Z (일반적으로 100차원의 균등 분포 또는 정규 분포에서 샘플링)
- 구조
  - 첫 층: 입력 Z를 완전 연결층(행렬 곱셈)으로 처리한 후, 4차원 텐서로 재구성합니다.
  - 중간 층: 여러 개의 fractional-strided(업샘플링) 컨볼루션 층을 사용하여 점차 해상도를 높여갑니다.
  - 출력층: Tanh 활성화 함수를 사용해 최종 이미지를 생성합니다.

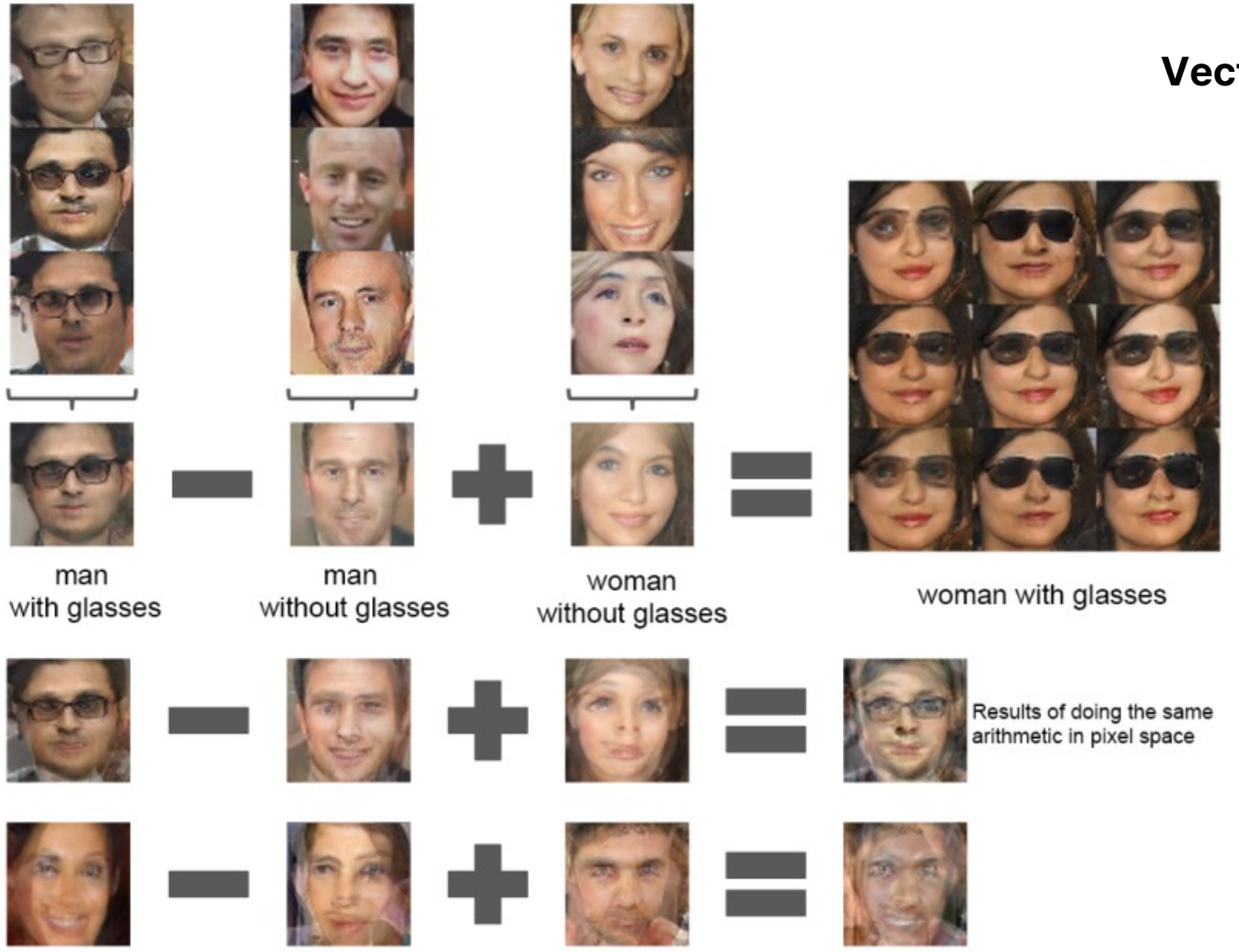
판별자(Discriminator) D:

- 입력: 실제 이미지 또는 생성된 이미지
- 구조
  - 여러 층의 스트라이드 컨볼루션을 통해 이미지의 특징을 추출합니다.
  - 마지막 층: Flatten한 후 단일 시그모이드 출력으로 실제 이미지인지 생성된 이미지인지를 판단합니다.

Walking in the latent space



Vector arithmetic for visual concepts



학습 결과 및 활용 사례

- 이미지 생성:
  학습 과정에서 생성자는 잠재 공간 Z 상에서 부드럽게 변화하는 이미지들을 생성하며, 이는 모델이 의미 있는 이미지 표현을 학습했음을 보여줍니다.
- 표현 학습:
  판별자에서 추출한 특징들을 다른 지도 학습 작업(예: CIFAR-10, SVHN 분류)에 활용하여 좋은 성능을 보였습니다.
- 잠재 공간 조작:
  생성자의 잠재 벡터에 대해 벡터 산술 연산을 수행하면, 이미지의 의미론적 속성(예: 얼굴의 자세, 창문의 유무 등)을 조작할 수 있음을 확인할 수 있습니다.

의의:

- 비지도 학습에 강점:
  라벨 없는 대규모 데이터셋으로부터 유용한 이미지 표현을 학습할 수 있어, 지도 학습 전 단계로 활용 가능함.
- 안정적인 학습:
  아키텍처 제약(스트라이드 컨볼루션, 배치 정규화, 활성화 함수 선택 등)을 통해 기존 GAN의 불안정한 학습 문제를 크게 개선함.
- 다양한 응용 가능성:
  이미지 생성, 특징 추출 및 표현 학습, 잠재 공간의 조작 등 다양한 응용 분야에 적용할 수 있음

한계 :

- 모델 불안정성:
  긴 학습 시간 동안 일부 필터가 단일 모드로 수렴하는 등 여전히 불안정성이 남을 수 있음.
- 세밀한 조작의 어려움:
  잠재 공간에서 의미 있는 벡터 산술을 수행하는 경우에도, 모든 의미론적 속성이 선형적으로 표현되지 않을 수 있음.
- 실제 응용의 한계:
  아직은 생성된 이미지의 해상도나 품질 면에서 상용 애플리케이션에 바로 적용하기에는 한계가 존재함.

02

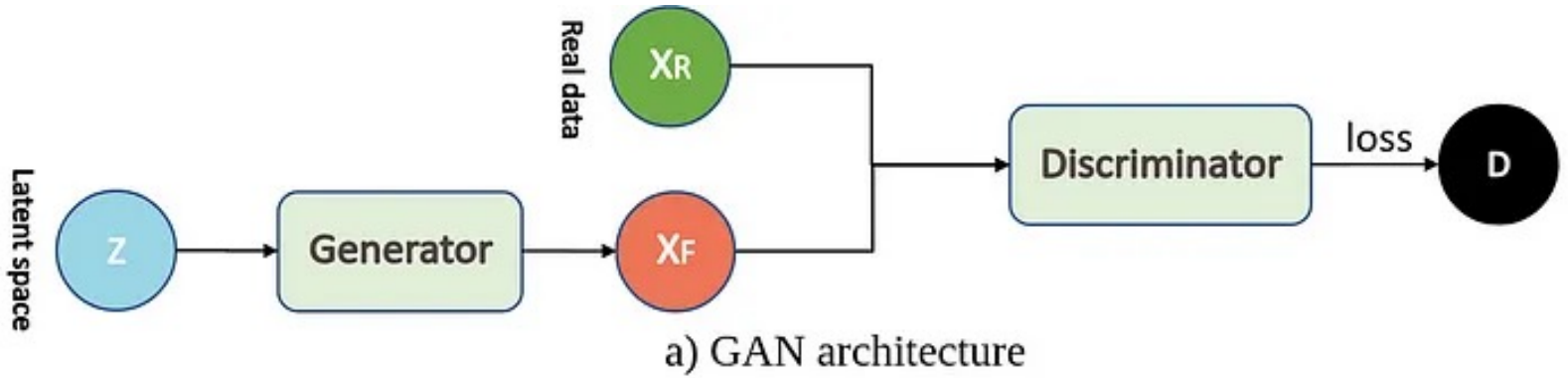
GAN 응용

Conditional GAN

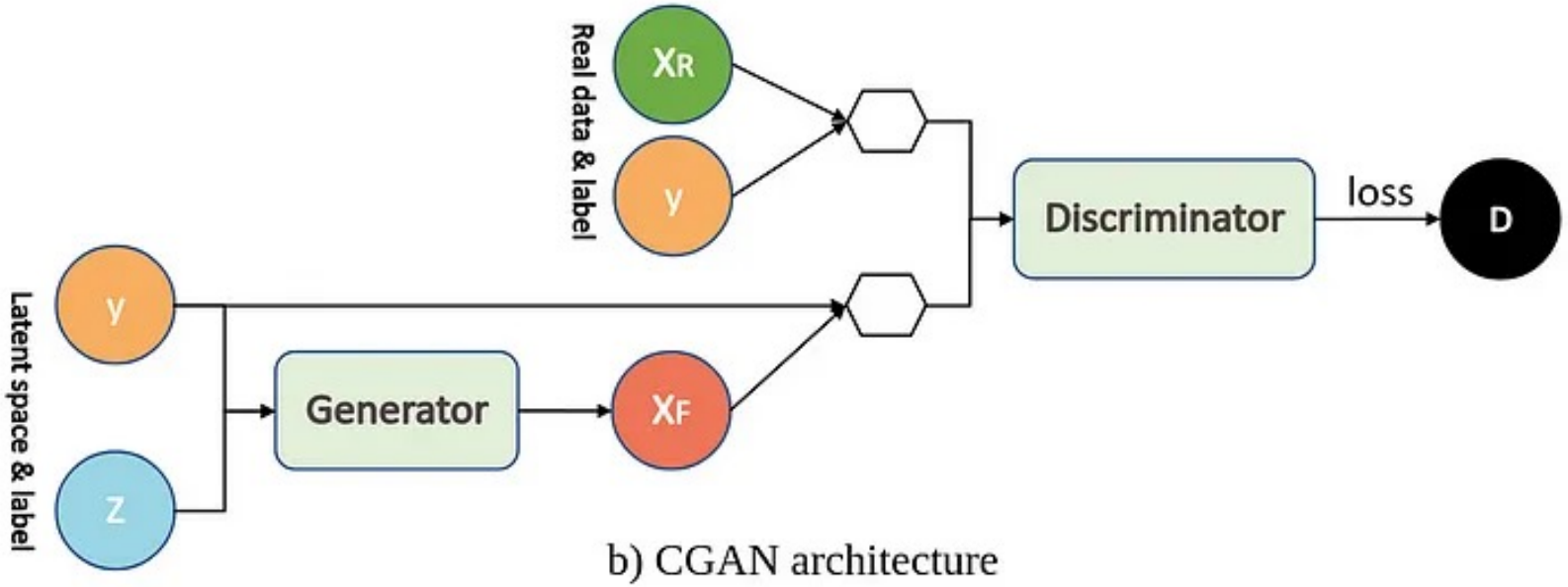
조건부 적대적 생성 네트워크 (Conditional GAN, cGAN)는 기존 GAN의 개념을 확장한 모델입니다.

핵심 아이디어

- 생성자(Generator)와 판별자(Discriminator) 두 네트워크가 서로 경쟁하는 방식으로 학습하는 GAN의 구조에서, cGAN은 이 두 네트워크 모두에 추가 정보(예: 클래스 레이블, 이미지 특징 등)를 입력으로 제공함으로써 생성 과정을 제어할 수 있게 만듭니다.
- 원하는 속성을 가진 데이터를 생성할 수 있으며, 다양한 조건(텍스트, 이미지, 레이블 등)에 따라 다채로운 결과물을 만들 수 있습니다.
- **Conditional Generative Adversarial Nets** https://arxiv.org/pdf/1411.1784



a) GAN architecture



b) CGAN architecture

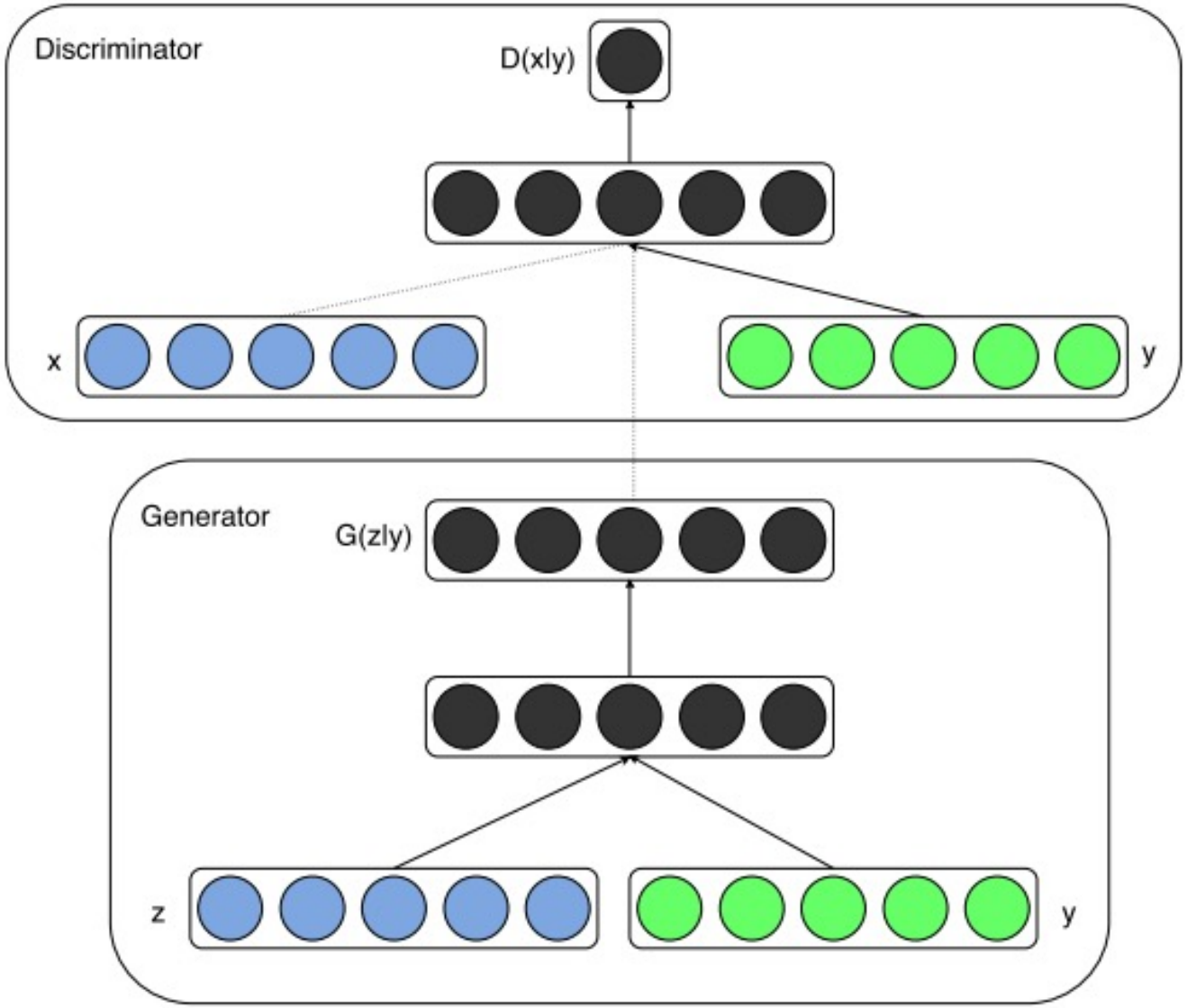
https://medium.com/@ma.bagheri/a-tutorial-on-conditional-generative-adversarial-nets-keras-implementation-694dcafa6282

모델 구성

- **조건 정보 y**
  - cGAN에서는 생성 및 판별 과정에 조건 정보를 함께 입력합니다.
  - 이 조건은 클래스 레이블, 텍스트, 이미지의 일부 정보 등 다양한 형태가 될 수 있습니다.
    (예: MNIST 숫자 생성의 경우, 조건은 '0'부터 '9'까지의 숫자 레이블)

- **생성자 G**
  - 잡음 벡터 z와 조건 y를 입력 받아, 이를 결합한 후 실제와 유사한 데이터를 생성합니다.
  - 목표는 판별자가 생성한 데이터를 실제 데이터와 구분하지 못하도록 만드는 것입니다.

- **손실값 모니터링의 어려움**
  - 입력으로 주어진 데이터 x와 조건 y를 받아, 해당 데이터가 실제 데이터인지 생성자가 만든 데이터인지 판별합니다.
  - 목표는 조건 정보에 맞지 않는 가짜 데이터를 정확히 구별하는 것입니다.


Discriminator
D(xly)
x
y
Generator
G(zly)
z
y

목적 함수

- **미니 맥스 게임**
  cGAN은 생성자와 판별자가 서로 경쟁하는 미니맥스 게임을 통해 학습됩니다.

cGAN

$$\min_G \max_D V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\boldsymbol{x})}[\log D(\boldsymbol{x}|\boldsymbol{y})] + \mathbb{E}_{\boldsymbol{z}\sim p_z(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z}|\boldsymbol{y})))].$$

GAN

$$\min_G \max_D V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim p_z(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))].$$

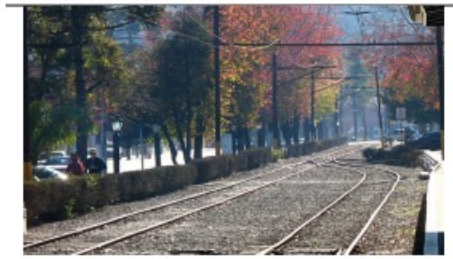
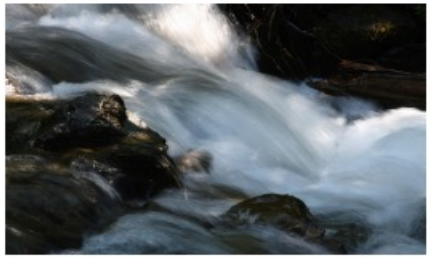
	User tags + annotations	Generated tags
	montanha, trem, inverno, frio, people, male, plant life, tree, structures, transport, car	taxi, passenger, line, transportation, railway station, passengers, railways, signals, rail, rails
	food, raspberry, delicious, homemade	chicken, fattening, cooked, peanut, cream, cookie, house made, bread, biscuit, bakes
	water, river	creek, lake, along, near, river, rocky, treeline, valley, woods, waters
	people, portrait, female, baby, indoor	love, people, posing, girl, young, strangers, pretty, women, happy, life

Table 2: Samples of generated tags



Figure 2: Generated MNIST digits, each row conditioned on one label

장점

- 조건 기반 제어:원하는 속성(예: 특정 클래스, 이미지 스타일 등)에 따라 데이터를 생성할 수 있어, 결과물의 다양성과 제어력이 높습니다.
- 다양한 응용 분야:이미지 생성, 텍스트-이미지 변환, 이미지 태깅, 데이터 보강 등 여러 분야에 활용 가능.
- 상대적으로 단순한 구조:복잡한 확률 분포를 모델링하는 대신, 두 네트워크가 경쟁하는 방식으로 학습되어 구현이 비교적 간단합니다.

한계

- 학습의 불안정성:GAN 계열 모델은 생성자와 판별자의 균형을 맞추기가 어려워 학습이 불안정할 수 있으며, 모드 붕괴(mode collapse) 문제도 발생할 수 있습니다.
- 조건 정보의 한계:조건 정보를 어떻게 효과적으로 결합할지에 대한 설계가 중요하며, 부적절한 조건 결합은 결과의 질 저하로 이어질 수 있습니다.
- 복잡한 응용에의 제한:단순한 조건부 생성에는 효과적이지만, 매우 복잡한 다중 모달 문제나 고차원 조건 정보의 경우 추가 연구와 개선이 필요합니다.

Pix2Pix

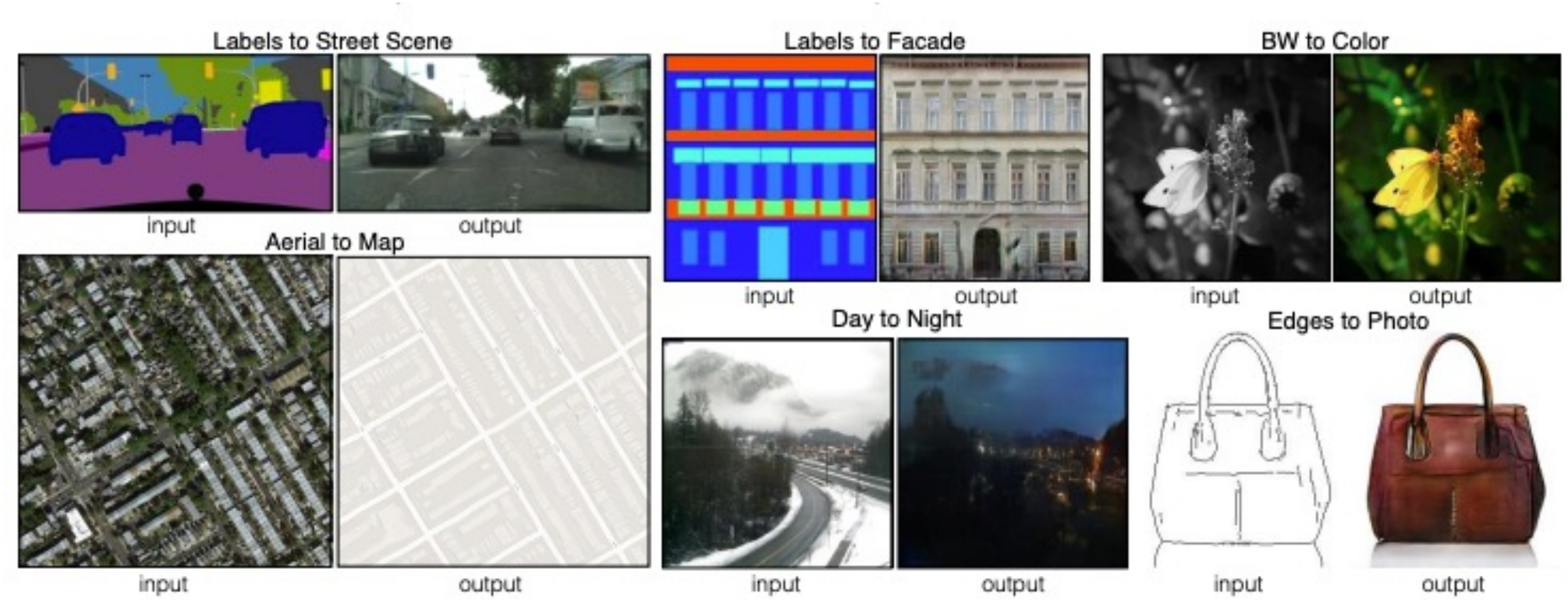
Pix2Pix는 이미지-투-이미지 번역 문제를 해결하기 위해 제안된 조건부 적대적 네트워크(cGANs) 기반의 모델입니다.

- 이미지-투-이미지 번역: 한 도메인의 이미지를 다른 도메인의 이미지로 변환하는 작업 (예: 스케치를 사진으로, 지도 이미지를 항공사진으로 변환 등).
- 조건부 GAN: 생성자(Generator)와 판별자(Discriminator)가 함께 학습하며, 생성자는 입력 이미지를 바탕으로 현실적인 출력을 만들어내고, 판별자는 생성된 이미지와 실제 이미지를 구분하는 역할을 수행합니다.
- Pix2Pix의 핵심은 "픽셀 대 픽셀" 매핑을 통해 복잡한 이미지 변환 문제를 단일 통합 모델로 해결할 수 있다는 점입니다. 이를 통해 다양한 응용 분야에서 손쉽게 활용할 수 있으며, 많은 창의적 프로젝트의 기초로 자리잡았습니다.
- **Image-to-Image Translation with Conditional Adversarial Network**
  https://arxiv.org/pdf/1611.07004 / https://phillipi.github.io/pix2pix/

Image Translation

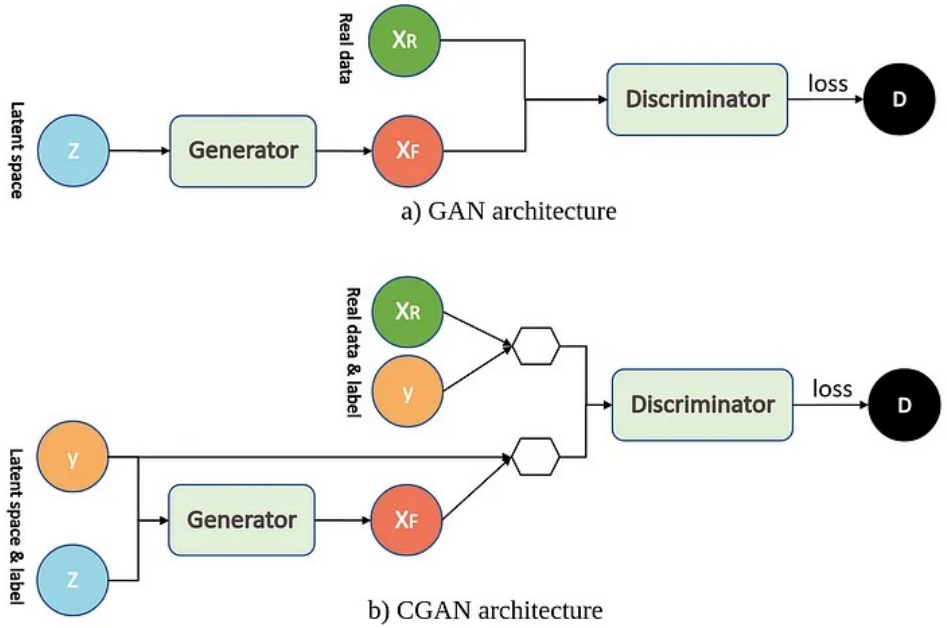
한 도메인(예: 스케치, 라벨 맵)의 이미지를 다른 도메인(예: 실제 사진, 컬러 이미지)으로 변환하는 작업을 의미합니다.

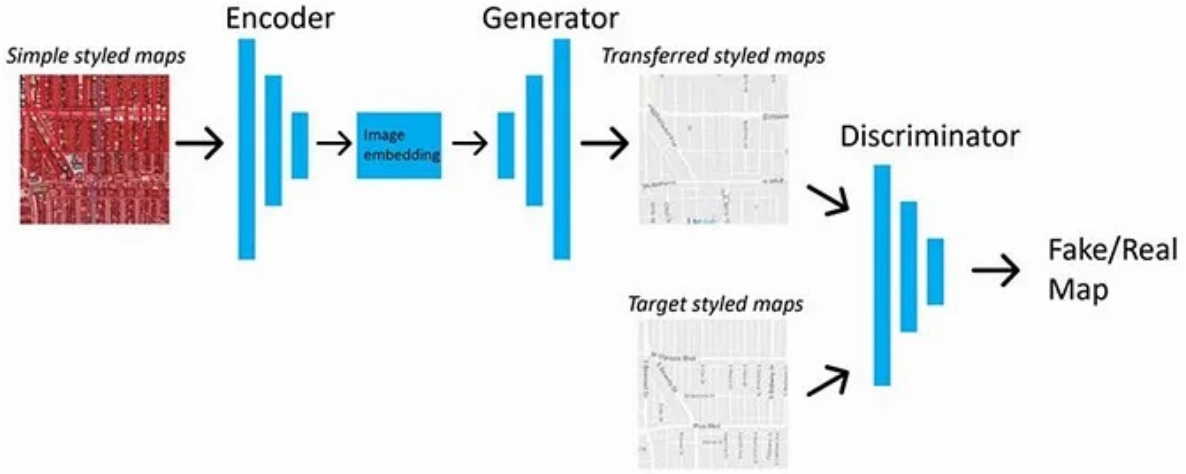
변환 과정에서 원본 이미지의 "의미론적 구조"는 유지하면서, 스타일이나 시각적 특성을 목표 도메인에 맞게 바꿉니다.



Main Idea

- **조건부 GAN(Conditional GAN) 기반**
  - Pix2Pix는 GAN(Generative Adversarial Network)을 확장하여, 입력 이미지 x와 짝지어진(target) 이미지 y가 함께 주어지는 상황을 가정합니다.
  - 각 학습 샘플은 (x,y)형태로, x는 "소스 도메인"의 이미지, y는 "타깃 도메인"의 이미지를 의미합니다.

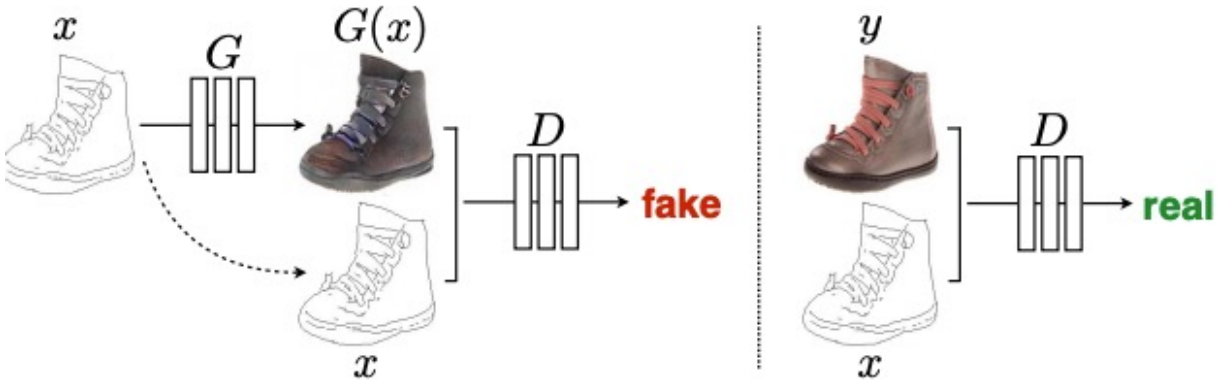




https://www.researchgate.net/figure/Data-flow-of-Pix2Pix-in-this-research_fig2_332932603

조건부 GAN(Conditional GAN) 기반

- Pix2Pix는 GAN(Generative Adversarial Network)을 확장하여, 입력 이미지 x와 짝지어진(target) 이미지 y가 함께 주어지는 상황을 가정합니다.
- 각 학습 샘플은 (x,y)형태로, x는 "소스 도메인"의 이미지, y는 "타깃 도메인"의 이미지를 의미합니다.
- **생성자 G**
  - 입력 이미지 x를 받아, 타깃 도메인 스타일로 변환된 결과 $\hat{y} = G(x)$를 생성합니다.
- **판별자 D**
  - (조건부인 경우) (x,y)혹은 (x, $\hat{y}$)쌍을 입력받아, (x,y)가 실제(real)인지, (x, $\hat{y}$)가 가짜(fake)인지를 구분합니다.



Objective Function

Pix2Pix는 크게 두 가지 손실을 결합하여 학습합니다.
(λ는 두 손실 항 사이의 가중치를 조절)

$$\min_G \max_D \; L_{cGAN}(G, D) + \lambda L_{L1}(G)$$

적대적 손실(Adversarial Loss)

- 생성자는 log(1−D(x,G(x,z)))를 최소화(혹은 log D(x,G(x,z)))를 최대화)하도록 학습하여, 판별자를 속이려 합니다.
- 판별자는 log D(x,y)를 최대화하여, 실제-가짜 구분 능력을 높이려 합니다.

$$\min_G \max_D \; \mathbb{E}_{(x,y)}[\log D(x, y)] + \mathbb{E}_{(x,z)}[\log(1 - D(x, G(x, z)))]$$

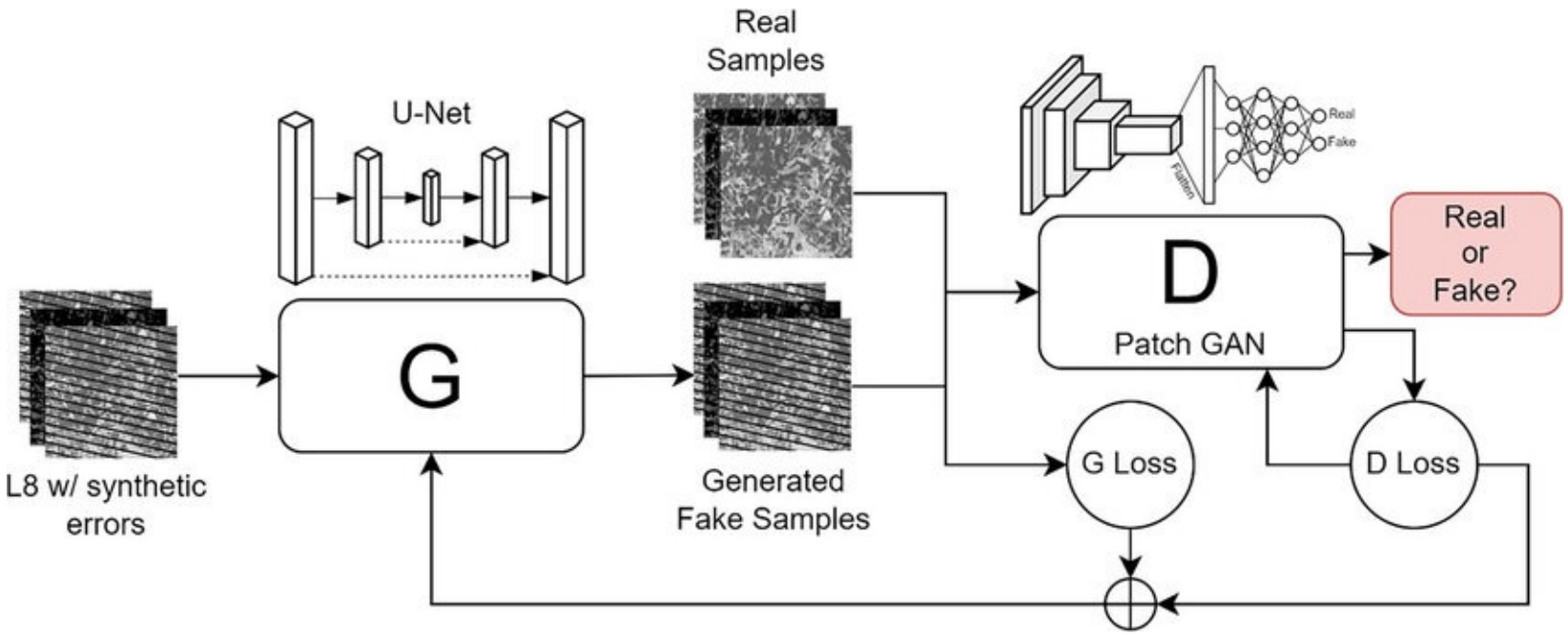
L1(혹은 L2) 회귀 손실(Regression Loss)

- 생성된 이미지 $\hat{y}$ =G(x)가 실제 이미지 y와 픽셀 단위로 유사해지도록 유도합니다.
- 전역적 구조 유지에 도움을 주며, 흐릿한 결과가 나올 수 있으나, 적대적 손실과 결합하면 선명도를 향상시킵니다.

$$L_{L1} = \mathbb{E}_{(x,y)}\left[\|y - G(x)\|_1\right]$$

U-Net 기반 생성자 (Skip Connections 활용)

- 인코더: 입력 이미지를 여러 단계의 컨볼루션(합성곱)으로 다운샘플링
- 디코더: 업샘플링을 통해 원래 크기로 복원스킵
- 연결: 인코더의 각 단계 출력을 대응되는 디코더 단계와 연결(concatenate)하여, 낮은 수준의 텍스처 및 위치 정보를 그대로 전달



https://www.researchgate.net/figure/Schema-of-Pix2Pix-GAN-architecture_fig3_383558465

PatchGAN 판별자 (N×N 패치 단위 평가)

- 전체 이미지를 하나로 보는 대신, 이미지를 N×N 패치로 나누어 각 패치별로 진짜/가짜를 판단.
- 16×16, 70×70 등 다양한 패치 크기를 시도할 수 있으며, 보통 70×70이 많이 사용됨.
- 국소적 텍스처 정보를 잘 포착하여, 생성된 이미지가 세밀한 영역에서도 사실적으로 보이도록 유도
- 전체 이미지를 평가하는 것보다 모델 파라미터 수가 적고, 학습이 효율적

Discriminator receptive field	Per-pixel acc.	Per-class acc.	Class IOU
1×1	0.39	0.15	0.10
16×16	0.65	0.21	**0.17**
70×70	**0.66**	**0.23**	**0.17**
286×286	0.42	0.16	0.11



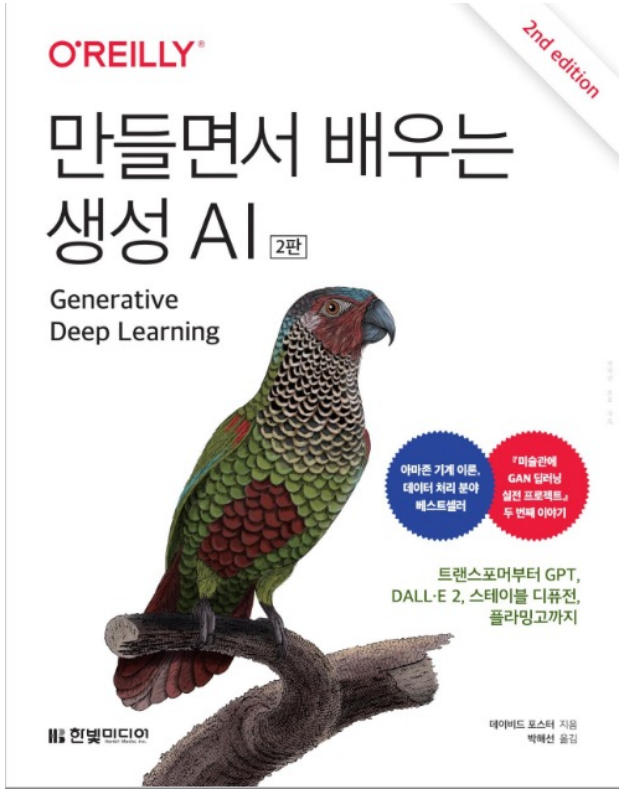
장점

- 범용성: 하나의 통합된 모델로 다양한 이미지 변환 문제에 적용할 수 있음
- 자동 손실 학습: 조건부 적대적 학습을 통해 데이터에 맞는 손실 함수를 자동으로 학습하므로, 복잡한 수작업 설계가 필요 없음
- 세밀한 결과물: U-Net 기반 생성자와 PatchGAN 판별자의 결합으로 고해상도의 세밀한 텍스처와 구조 정보를 유지한 이미지를 생성함.

한계

- 학습의 불안정성: GAN 특성상 학습 과정이 불안정할 수 있으며, 하이퍼파라미터 조정이 중요함
- 흐릿한 결과: L1 손실만 사용하면 결과가 흐릿할 수 있으나, 적대적 손실과 결합해야 효과적임
- 출력의 다양성: 생성된 결과가 결정론적일 수 있으며, 무작위성을 완전히 포착하기 어려울 수 있음.

참고자료



- VAE : https://arxiv.org/pdf/1312.6114
- GAN : https://arxiv.org/pdf/1406.2661
- DCGAN : https://arxiv.org/pdf/1511.06434
- cGAN : https://arxiv.org/pdf/1411.1784
- Pix2Pix : https://arxiv.org/pdf/1611.07004
- 스탠포드 cs231n : http://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture13.pdf
- 서울대 시각적 이해를 위한 머신러닝 : https://www.youtube.com/watch?v=MuTibPS09wQ&list=PL0E_1UqNACXDTwuxUzCl5AeEjXBfWxCwc&index=21

감사합니다 ☺